Joint Service Deployment and Task Scheduling for Satellite Edge Computing: A Two-Timescale Hierarchical Approach

Qinqin Tang, Renchao Xie, *Senior Member, IEEE*, Zeru Fang, Tao Huang, *Senior Member, IEEE*, Tianjiao Chen, Ran Zhang, and F. Richard Yu, *Fellow, IEEE*

***Abstract*— In this paper, we establish a two-timescale framework for the joint service deployment and task scheduling problem in satellite edge computing networks. We aim to optimize the computing performance of networks with diverse quality-of-service (QoS) guarantees for computing tasks. Specifically, to capture the small-timescale network dynamics and task randomness, we formulate the task scheduling problem as a constrained Markov decision process (CMDP) to minimize the energy consumption, load imbalance and packet loss of networks while ensuring the long-term delay. The Lyapunov technique is employed to deal with the delay constraints. A soft actor-critic (SAC)-based deep reinforcement learning (DRL) framework is designed to learn the stationary scheduling policy. We further explore the significant impact of deploying diverse services on the performance of task scheduling in satellite edge computing. Considering that frequent deployment of services will incur huge deployment overhead, we optimize the service deployment on a larger timescale. The optimization problem is modeled as an integer programming problem to improve the service capability of networks and reduce service deployment costs. A heuristic-based atomic orbital search (AOS) approach is proposed to obtain the superior policy with low complexity. Due to the correlation between the problems of two timescales, a hierarchical solution is constructed to iteratively find the excellent solution. Finally, extensive simulations are conducted to validate the effectiveness and superiority of the proposed scheme.**

***Index Terms*— Satellite edge computing, task scheduling, service deployment, two-timescale hierarchical framework.**

Manuscript received 30 October 2023; revised 15 November 2023; accepted 15 December 2023. Date of publication 14 February 2024; date of current version 9 May 2024. This work was supported in part by the Natural Science Foundation of Beijing under Grant 4244067, in part by the National Natural Science Foundation of China under Grant 62171046, and in part by the China Postdoctoral Science Foundation under Grant 2022TQ0048 and Grant 2022M720519. *(Corresponding author: Renchao Xie.)*

Qinqin Tang and Zeru Fang are with the State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications, Beijing 100876, China (e-mail: qqtang@bupt.edu.cn; zerufang01@gmail.com).

Renchao Xie, Tao Huang, and Ran Zhang are with the State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications, Beijing 100876, China, and also with the Purple Mountain Laboratories, Nanjing 211111, China (e-mail: Renchao_xie@bupt.edu.cn; htao@bupt.edu.cn; zhangran@bupt.edu.cn).

Tianjiao Chen is with the China Mobile Research Institute, Beijing 100053, China (e-mail: chentianjiao@chinamobile.com).

F. Richard Yu is with the Guangdong Laboratory of Artificial Intelligence and Digital Economy (Shenzhen), Shenzhen 518107, China, and also with the School of Information Technology, Carleton University, Ottawa, ON K1S 5B6, Canada (e-mail: richard.yu@carleton.ca).

Color versions of one or more figures in this article are available at https://doi.org/10.1109/JSAC.2024.3365889.

Digital Object Identifier 10.1109/JSAC.2024.3365889

I. Introduction

THE sixth generation (6G) mobile network is designed to provide global coverage to ensure Internet availability and seamless access anytime and anywhere [1]. As traditional terrestrial networks are limited by bottlenecks such as geographic location and economic costs, satellite networks (especially low earth orbit (LEO) satellite networks) are potential technologies to realize this vision due to their seamless coverage and low transmission delay [2], [3]. In recent years, with the vigorous development of the Internet of things (IoT), a large number of IoT services have emerged in remote areas, such as disaster monitoring, marine transportation, object identification and tracking, etc., and the concept of the Internet of remote things (IoRT) is emerging [4]. IoRT users are generally constrained in terms of computing power and battery power, making it difficult to cope with the surge in computing tasks. An effective way is to offload IoRT tasks to a more powerful cloud server on the ground for further execution through satellite relay transmission [5]. However, due to the limitations of the physical location of satellite networks, satellite relay transmission will bring a non-negligible long transmission distance and heavy transmission load, which is usually intolerable for IoRT services with low latency requirements.

As an emerging computing paradigm, edge computing can provide users with computing resources in close proximity by extending cloud computing platforms to the edge of the network or even to the mobile devices themselves [6], [7], [8]. Therefore, by introducing edge computing to satellite networks, satellite edge computing can further enhance satellite transmission rate and reduce processing delay, thus effectively improving the computing performance of tasks [9], [10], [11]. In satellite edge computing, computing resources are ubiquitously distributed and limited on a single satellite. As the amount of data offloaded by IoRT users increases, computing requirements may exceed the processing capabilities of a single satellite, resulting in prolonged computing response times and high power consumption. In addition, the distribution of computing tasks in satellite edge computing networks is uneven and changes with the high-speed movement of satellites, leading to inefficient and imbalanced utilization of network resources.

Task scheduling is a promising approach to address the above challenges. Through the efficient task scheduling of

satellite edge computing, tasks can be offloaded to computing nodes with more available resources for processing via edge cooperation and cloud-edge cooperation according to their diverse characteristics, thus improving the computing performance of tasks and optimizing resource utilization. In this architecture, a lot of work has been carried out to optimize the cost [12], delay [13], [14], energy consumption [15], [16], etc. in the task computing process by determining the task scheduling policy. These existing works usually formulate the task scheduling problem as a one-shot optimization problem. However, due to the dynamic nature of satellite edge computing networks, task scheduling decisions need to be determined over sequential scheduling slots to achieve long-term computing performance optimization. Solving one-shot optimization problems over small-timescale scheduling slots is computationally intractable.

On the other hand, how diverse services are deployed in satellite edge computing networks will affect the processing performance of computing tasks by influencing task scheduling policies, which has not been fully considered in existing work. To utilize satellite edge computing resources to provide a variety of computing services, corresponding services need to be deployed at satellites. Due to the limitation of storage capacity, a single satellite cannot deploy all services, and its computing requests vary with time and space. Consequently, a static service deployment policy is not feasible, leading to inefficient service performance of satellite edge computing networks. However, a significant deployment overhead is incurred if the service deployment policy is frequently adjusted along with task scheduling. Therefore, service deployment needs to be optimized on a larger timescale to increase the service capacity of satellite edge computing networks and reduce the cost of service deployment.

Since the two decisions (i.e., service deployment and task scheduling) are coupled, this paper proposes a two-timescale service deployment and task scheduling framework to maximize the service performance of satellite edge computing networks. Specifically, our significant contributions are summarized as follows.

- We establish a two-timescale hierarchical framework for the joint service deployment and task scheduling problem in satellite edge computing networks. Our objective is to optimize the computing performance of networks with diverse quality-of-service (QoS) guarantees for computing tasks.
- On small timescales, to capture the network dynamics and task randomness of satellite edge computing networks, we formulate the task scheduling problem as a constrained Markov decision process (CMDP), whose goal is to optimize the energy consumption, load balancing and packet loss of satellite edge computing networks while ensuring the long-term delay constraints. The CMDP is transformed into MDP by the Lyapunov optimization technique, and the delay deficit queue is constructed to characterize the satisfaction of long-term delay constraints. A soft actor-critic (SAC)-based deep reinforcement learning (DRL) framework is designed to learn a stationary scheduling policy.
- On large timescales, the service deployment problem is modeled as an integer programming problem to optimize the computing service capacity of satellite edge computing networks and reduce the cost of service deployment. A heuristic-based atomic orbital search (AOS) approach is proposed to obtain the superior service deployment policy with low complexity.
- A hierarchical solution is constructed to iteratively find the solution to the two-timescale joint optimization problem. The performance of our proposed scheme is evaluated through extensive simulations. Numerical results demonstrate the effectiveness and superiority of our proposal compared to the benchmark schemes in optimizing satellite network performance and ensuring computing QoS.

The rest of this paper is organized as follows. In Section II, we delve into an exploration of related studies. Section III elaborates on the design of the satellite edge computing system. In Section IV, we introduce the framework for joint service deployment and task scheduling. To tackle the two-timescale joint optimization issue, we propose a hierarchical solution that integrates DRL and AOS approaches, which is discussed in Section V. Section VI presents the numerical results derived from our experiments, and finally, we draw our conclusions in Section VII.

II. Related Work

Advances in mega-LEO satellite constellations such as Starlink [17], OneWeb [18] and Kuiper [19] have facilitated the growth of satellite data and applications. Edge computing is regarded as a vital technological catalyst that significantly enhances network transmission speeds and reduces processing latency [6], [7], [20]. The integration of edge computing with satellite networks has also drawn substantial interest. In [21], Zhang et al. explored methods for implementing mobile edge computing (MEC) within satellite-terrestrial networks to enhance the QoS. They designed a satellite MEC (SMEC) framework enabling users close to MEC servers to access MEC services via satellite links. In [22], Ding et al. examined a dense satellite-terrestrial integrated MEC network architecture that aims to offload computations from ground user terminals to terrestrial edge servers with the aid of satellites. In [23], a joint mechanism was developed to allocate computing resources effectively among virtual machines (VMs) in space-air-ground integrated networks. Additionally, a reinforcement learning-based computing offloading algorithm was designed to adapt to dynamic network conditions.

Satellite edge computing is anticipated to offer on-board data processing, which would save valuable downlink communication resources, and facilitate tasks requiring low latency and high reliability [9], [10], [11]. Task scheduling emerges as a crucial aspect in satellite edge computing, as it can enhance the processing performance of computing tasks and optimally leverage the finite resources of satellite networks. To fully utilize the computing resources of satellite constellations as well as to reduce the computing cost of ground users offloading tasks to satellite networks, Zhang et al. proposed a greedy

orbital edge computing (OEC) task allocation algorithm in [12] to cope with the limited computing resources in satellite constellations. In [13], Wan et al. introduced a data-driven fair hierarchical scheduling (DFHS) in space-ground integrated IoT. This methodology accomplishes equitable scheduling of packets with varying service types and lengths, significantly reducing the average packet transmission delay. Cassará et al. [14] leveraged the compute-as-a-service capabilities of satellites to build on-orbit compute continuums, enabling equal computing access and optimizing computing delay, CPU utilization, and data rates for mega-LEO satellite constellations. Cao et al. [16] presented an optimal offloading framework for LEO satellite edge systems, which conceptualizes an optimal offloading access scheme and policy to improve the performance of satellite-terrestrial integrated networks (STINs) in terms of computing latency, energy efficiency and coverage. Applying deep learning methods to improve the efficiency of task scheduling decisions in edge computing systems has become promising [24]. Zhang et al. [25] developed an intelligent computing offloading scheme based on the deep deterministic policy gradient (DDPG) to adapt to the dynamic task arrival queue environment of satellite edge computing. In [26], Ji et al. proposed a multi-agent double actors twin delayed deterministic policy gradient (MA-DATD3) algorithm to solve the offloading optimization problem of the LEO layer in digital twin-empowered satellite-ground collaborative edge computing networks.

Given the constrained resources of satellites, the optimized deployment of satellite edge computing services is a technical issue of concern. However, this issue has not been fully considered in existing research. Service deployment is not new to terrestrial edge computing. For instance, Nguyen et al. [27] introduced a decentralized protocol for MEC service deployment and discovery. This system, based on a revised version of content-centric networking, assists in deploying services and identifying services in neighboring nodes, aiming for low transmission delay and control overhead. Wang et al. [28] analyzed the redundant deployment of intricate service-based applications within edge environments. Their aim was to minimize network latency and data transfer expenses while adhering to the constraints of the deployment budget. Li et al. [29] focused on robustness-oriented edge application deployment, with the goal of optimizing the collective robustness for all application users across each respective region. Computing tasks in highly dynamic satellite edge computing scenarios vary not only over time but also over space, so these works are not directly applicable.

Previous research on task scheduling for satellite edge computing primarily focuses on one-time optimizations of various objectives, leaving the one-shot optimization problems over small-timescale scheduling slots largely unaddressed. Additionally, the dynamic service deployment aspect in satellite edge computing has not been fully explored to enhance task scheduling performance. To bridge this gap, this work presents a novel two-timescale hierarchical framework that jointly optimizes service deployment and task scheduling for satellite edge computing. By facilitating continuous interplay between small-timescale task scheduling and large-timescale service deployment problems, the overall performance of the system is significantly improved.

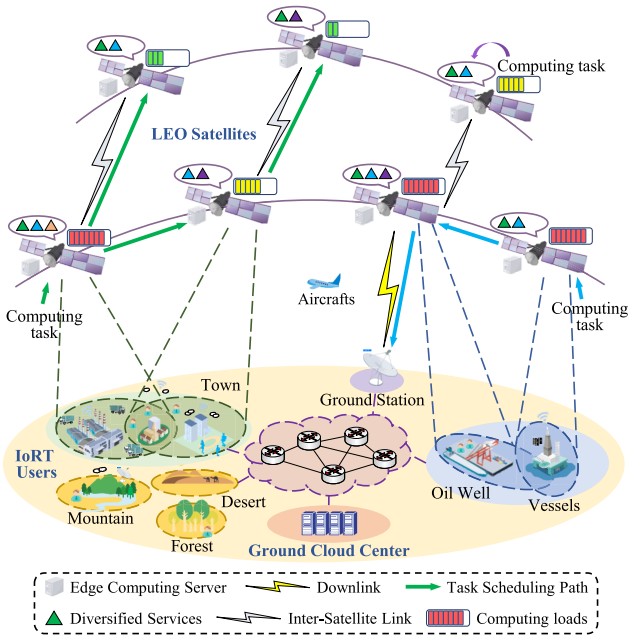


Fig. 1. Illustration of the satellite edge computing network.

III. System Description

In this section, we give a broad overview of the satellite edge computing system model under consideration in our research.

A. System Model

As depicted in Fig. 1, we focus on a satellite edge computing network that comprises three main components: LEO satellites equipped with edge computing capabilities, IoRT users, and the ground cloud center. The system functions in a time-slotted manner, dividing time into fixed-duration slots, represented as Δt. The time slot index is designated by t, where $t \in \mathcal{T} = \{1, \ldots, T\}$. Drawing on numerous prior studies [30], [31], [32], we work under the assumption of a quasistatic scenario. This implies that while the environment remains static within each time slot, it may vary between different slots. Denote by $\mathcal{N} = \{1, 2, \ldots, N\}$ the set of LEO satellites. Denote by $\mathcal{J} = \{1, 2, \ldots, J\}$ the set of services. In this study, we assume that services operate in a containerized environment. For a specific type of computing task, it can only be computed and executed by the computing node that has the corresponding service deployed. It's important to note that, in this work, services refer to configured applications. For instance, if there is a computing task seeking computing services for image recognition in the satellite edge computing network, the necessary application (service) must be pre-deployed on the designated LEO satellite. This application is configured with various files necessary to provide image recognition functions. Owing to resource constraints in satellite edge computing networks, the computing resources of LEO satellites are significantly lower than those of the ground cloud center. As a result, each LEO satellite can only deploy a subset of services $\mathcal{J}_n \subseteq \mathcal{J}$, while the ground cloud

center can host the entire set of services $\mathcal{J}$. In a containerized operating environment, each deployed (configured) service is designed to handle one computing task at a time. To enhance processing efficiency, a service can be deployed with multiple replicas on a computing node, facilitating parallel processing. The number of replicas for service $j \in \mathcal{J}$ in the network is represented by v_j. Additionally, the storage capacity required for deploying service $j \in \mathcal{J}$ on an LEO satellite is denoted by z_j. We use $v_{j,n}$ to represent the number of replicas of service $j \in \mathcal{J}_n$ deployed on LEO satellite n, with the condition that $\sum_{n\in\mathcal{N}} v_{j,n} = v_j$ holds. The computing resource (quantified in CPU cycles) required to execute one bit of tasks for service j is denoted by c_j. The computing capability of each service replica is closely tied to the capabilities of the computing nodes on which it is deployed. The computing capability of service replica j deployed on LEO satellite n is $f_{n,j}$.

Computing tasks generated on LEO satellites originate from IoRT users. The task generation state of LEO satellite n is denoted by $\Gamma_n(t) = \{\Gamma_{1,n}(t), \ldots, \Gamma_{|\mathcal{J}_n|,n}(t)\}$, where $\Gamma_{j,n}(t)$ is the set of computing tasks for service j generated at LEO satellite n in time slot t. Then, we can represent the task generation state of LEO satellites in time slot t as $\Gamma(t) = \{\Gamma_1(t), \Gamma_2(t), \ldots, \Gamma_N(t)\}$. We define $k_{j,n}(t) \in \Gamma_{j,n}(t)$ as the task in $\Gamma_{j,n}(t)$. Generated computing tasks (such as images, etc.) have varying task sizes. We denote by $x_{k,j,n}(t)$ the data size (in bits) of task $k_{j,n}(t)$. For service j, the processing of unit computing tasks has a tolerable delay, defined as D_j^{th}. To reduce the impact of limited satellite computing resources on QoS, inter-satellite and satellite-to-ground task scheduling will be carried out. We define $o^m_{k,j,n}(t) \in \{0, 1\}$ to represent the task scheduling decision of task $k_{j,n}(t)$. When task $k_{j,n}(t)$ is scheduled to LEO satellite m, then $o^m_{k,j,n}(t) = 1$, otherwise $o^m_{k,j,n}(t) = 0$. When task $k_{j,n}(t)$ is processed locally, we have $m = n$. In our study, we do not consider the scenario where tasks can be split, which means that each computing task can only be scheduled to one computing node for processing. To specify the data transmission path when scheduling task $k_{j,n}(t)$, we define $p_{k,j,n}(t) = \{n, r^1_{k,j,n}(t), r^2_{k,j,n}(t), \ldots, d_{k,j,n}(t)\}$, where n represents the source satellite, r represents the relay satellites, and d represents the destination computing node (i.e., satellite or ground data center). Note that if $o^m_{k,j,n}(t) = 1$, we have $d_{k,j,n}(t) = m$. Moreover, if $\sum_{m\in\mathcal{N}} o^m_{k,j,n}(t) = 0$, then the destination computing node is the ground cloud data center, and we have $d_{k,j,n}(t) = 0$.

B. Satellite Queue Model

LEO satellites leverage their computing resources to execute tasks generated locally as well as those received from other LEO satellites. Tasks that have arrived but have not yet been executed are stored in data queues. LEO satellites maintain a separate queue for each type of service. For LEO satellite n, it will manage $|\mathcal{J}_n|$ data queues for corresponding services, respectively. During time slot t, $\mathcal{Q}_{n,j}(t)$ represents the length of the queue for service $j \in \mathcal{J}_n$ at LEO satellite $n \in \mathcal{N}$. The queue state of LEO satellites in time slot t is denoted by $\mathcal{Q}(t) = \{\mathcal{Q}_1(t), \mathcal{Q}_2(t), \ldots, \mathcal{Q}_N(t)\}$, where $\mathcal{Q}_n(t) = \{\mathcal{Q}_{n,1}(t), \ldots, \mathcal{Q}_{n,|\mathcal{J}_n|}(t)\}$ is the queue state of LEO satellite n.

New arrival computing tasks include both locally generated tasks and those received from other LEO satellites. Therefore, the new arrival computing load (in bits) for service j on LEO satellite n in time slot t can be denoted by

$$\Lambda_{n,j}(t) = \sum_{m\in\mathcal{N}} \sum_{j\in\mathcal{J}} \sum_{k\in\Gamma_{j,m}(t)} o^n_{k,j,m}(t) x_{k,j,m}(t). \tag{1}$$

Thus, for each type of service $j \in \mathcal{J}_n$ at LEO satellite n, the queue evolves as follows:

$$\mathcal{Q}_{n,j}(t+1) = \min\left\{\mathcal{Q}_{n,j}(t) + \Lambda_{n,j}(t) - \Omega_{n,j}(t), \mathcal{Q}^{\max}_{n,j}\right\}. \tag{2}$$

where $\mathcal{Q}^{\max}_{n,j}$ represents the upper limit length of the data queue of service j at LEO satellite n. $\Omega_{n,j}(t)$ is the overall computing load of service j processed by LEO satellite n in time slot t, which is computed as $\Omega_{n,j}(t) = f_{n,j} v_{j,n} \Delta t / c_j$.

If the queue reaches its maximum capacity, some tasks may be dropped and lost. The amount of dropped tasks for LEO satellite n's queue j can be calculated by

$$\Psi_{n,j}(t) = \max\left\{\mathcal{Q}_{n,j}(t) + \Lambda_{n,j}(t) - \Omega_{n,j}(t) - \mathcal{Q}^{\max}_{n,j}, 0\right\}. \tag{3}$$

Here, a positive value for $\Psi_{n,j}(t) > 0$ signifies the occurrence of a queue overflow event at LEO satellites. The total quantity of dropped tasks across the satellite edge computing network in time slot t can be calculated as $\Psi(t) = \sum_{n\in\mathcal{N}} \sum_{j\in\mathcal{J}_n} \Psi_{n,j}(t)$.

C. Communication Model

The communication model includes the inter-satellite link communication model and the downlink communication model.

1) Inter-Satellite Link Communication Model: LEO satellites are interconnected using laser inter-satellite links (ISL). In time slot t, the data transmission rate (in bps) from LEO satellite n to LEO satellite m, $R_{n,m}(t)$, is determined as [33]:

$$R_{n,m}(t) = \frac{p_n G^{ts}_n G^{rs}_m L_{n,m}(t)}{h T_s \cdot \left(\frac{E_b}{N_0}\right)_{req} \cdot M}, \tag{4}$$

where $L_{n,m}(t)$ is the free space loss, and is expressed as:

$$L_{n,m}(t) = \left(\frac{c}{4\pi H_{n,m}(t) f}\right)^2, \tag{5}$$

Here, c is indicative of the speed of light, while f designates the communication center frequency of the ISL. The slope distance between LEO satellite n and LEO satellite m during time slot t is represented by $H_{n,m}(t)$. The term p_n corresponds to the transmission power of LEO satellite n. The transmitting antenna gain of LEO satellite n and the receiving antenna gain of LEO satellite m for the inter-satellite link are denoted by G^{ts}_n and G^{rs}_m, respectively. Furthermore, h stands for the Boltzmann constant, T_s defines the total system noise temperature, $\left(\frac{E_b}{N_0}\right)_{req}$ signifies the required received energy per bit relative to the noise density, and M indicates the link margin.

2) Downlink Communication Model: The calculation of the signal-to-noise ratio (SNR) for the downlink from LEO satellite n to the ground station during time slot t can be performed as [5]:

$$\mathrm{SNR}_n(t) = \frac{p_n G_n^{tg} G_0^{rg} L_{n,0}(t) L_n^p(t)}{N_0}, \tag{6}$$

where N_0 refers to the noise power, $L_n^p(t)$ is the rain attenuation, G_n^{tg} is the transmitting antenna gain of LEO satellite n to the ground station, G_0^{rg} is the receiving antenna gain of the ground station, $L_{n,0}(t)$ is the corresponding free space loss and can be calculated using (5).

Then, the attainable data rate (measured in bps) for the downlink from LEO satellite n to the ground station during the time slot t can be formulated as follows:

$$R_{n,0}(t) = W_0 \cdot \log_2\left(1 + \mathrm{SNR}_n(t)\right), \tag{7}$$

where W_0 is the available bandwidth.

3) Communication State: According to the inter-satellite link and downlink communication model, we define the communication state between LEO satellites in time slot t as $\mathcal{R}(t) = \{\mathcal{R}_1(t), \mathcal{R}_2(t), \ldots, \mathcal{R}_N(t)\}$, where $\mathcal{R}_n(t) = \{R_{n,0}(t), R_{n,1}(t), \ldots, R_{n,m}(t), \ldots, R_{n,N}(t)\}$ is the communication state between LEO satellite n and its adjacent LEO satellites or the ground station in time slot t. In this work, we employ the orthogonal frequency-division multiple access (OFDMA) scheme for transmitting computing tasks over both inter-satellite links and downlinks. Within a given time slot, frequency resources on a specific link are uniformly divided and allocated based on the computing tasks slated for transmission during that time slot.

D. Processing Delay Model

Computing tasks can be handled either locally on an LEO satellite or scheduled to other LEO satellites when the local satellite experiences a high computing load. In the considered system, the average processing delay mainly includes the data transmission delay (if any), the waiting delay at the data buffer and the execution delay. If LEO satellite n and LEO satellite m are not connected by direct link, they need to be forwarded through other LEO satellites. The transmission delay is obtained by accumulating the delay on each ISL, as shown in the equation below.

$$D_{k,j,n}^{\mathrm{com},m}(t) = \sum_{p,q \in p_{k,j,n}(t)} \frac{o_{k,j,n}^m(t) x_{k,j,n}(t) M_{p,q}(t)}{R_{p,q}(t)}. \tag{8}$$

Here, p and q represent consecutive satellite indexes during transmission. Moreover, $M_{p,q}(t)$ is the total number of computing tasks transmitted on the link from LEO satellite p to LEO satellite q in time slot t, which can be calculated as:

$$\begin{aligned} M_{p,q}(t) = &\sum_{n,m \in \mathcal{N}} \sum_{j \in \mathcal{J}} \sum_{k \in \Gamma_{j,n}(t)} o_{k,j,n}^m(t) \mathbf{1}\left(p, q \in p_{k,j,n}(t)\right) \\ &+ \sum_{n \in \mathcal{N}} \sum_{j \in \mathcal{J}} \sum_{k \in \Gamma_{j,n}(t)} \left(1 - \sum_{m \in \mathcal{N}} o_{k,j,n}^m(t)\right) \mathbf{1}\left(p, q \in p_{k,j,n}(t)\right), \end{aligned} \tag{9}$$

where $\mathbf{1}(\cdot)$ is an indicator function that evaluates to 1 if the case in parentheses is true, and 0 otherwise.

Upon arrival at an LEO satellite's data queue, a computing task needs to wait for its turn to be processed. The waiting time of a task is determined by the execution time of all the existing computing tasks that are already present in the satellite's data queue, which consists of two parts: first, the delay caused by processing the previously accumulated tasks in the queue, which can be given by [32]:

$$D_{k,j,n}^{\mathrm{blog},m}(t) = \frac{o_{k,j,n}^m(t) c_j \mathcal{Q}_{m,j}(t)}{f_{m,j} v_{j,m}}, \tag{10}$$

and second, the average waiting time experienced by new tasks before they are processed, which can be given by:

$$D_{k,j,n}^{\mathrm{wait},m}(t) = \frac{o_{k,j,n}^m(t) c_j q_{k,j,n}^m(t)}{2 f_{m,j} v_{j,m}}, \tag{11}$$

where $q_{k,j,n}^m(t) = \Lambda_{m,j}(t) - o_{k,j,n}^m(t) x_{k,j,n}(t)$ represents the aggregated computing load for service j on LEO satellite n in time slot t except task $k_{j,n}(t)$.

The task processing delay is primarily related to the computing resources required for the task and the computing capacities of LEO satellites, which can be calculated as:

$$D_{k,j,n}^{\mathrm{comp},m}(t) = \frac{o_{k,j,n}^m(t) x_{k,j,n}(t) c_j}{f_{m,j}}. \tag{12}$$

When computing tasks are directed to the ground cloud center for processing, the total processing delay incorporates the transmission delay from LEO satellites to the ground cloud center $D_{k,j,n}^{\mathrm{ccom}}(t)$ and the execution delay at the ground cloud center $D_{k,j,n}^{\mathrm{ccomp}}(t)$. These can be computed as follows:

$$D_{k,j,n}^{\mathrm{ccom}}(t) = \sum_{p,q \in p_{k,j,n}(t)} \frac{\left(1 - \sum_{m \in \mathcal{N}} o_{k,j,n}^m(t)\right) x_{k,j,n}(t) M_{p,q}(t)}{R_{p,q}(t)}, \tag{13}$$

and

$$D_{k,j,n}^{\mathrm{ccomp}}(t) = \frac{\left(1 - \sum_{m \in \mathcal{N}} o_{k,j,n}^m(t)\right) x_{k,j,n}(t) c_j}{f_0}. \tag{14}$$

Here, f_0 is for the computing ability of the ground cloud center. The destination of $p_{k,j,n}(t)$ is the ground cloud data center. The data transmission between the ground station and the ground cloud center takes place over a fixed wired link, which usually has a high transmission rate. Therefore, we ignore this transmission delay. Moreover, given the ample computing resources in the ground cloud center, the waiting time in the cloud buffer is overlooked.

Based on the above, the average unit task processing delay can be calculated as:

$$D_j(t) = \frac{1}{\sum_{n \in \mathcal{N}} |\Gamma_{j,n}(t)|} \sum_{n \in \mathcal{N}} \sum_{k \in \Gamma_{j,n}(t)} \frac{1}{x_{k,j,n}(t)}. \tag{15}$$

E. Energy Consumption Model

The energy consumption model primarily comprises energy consumption for task transmission and task processing. The energy consumed to transmit task $k_{j,n}(t)$ to LEO satellite m and to the ground station can be expressed as follows:

$$E_{k,j,n}^{\mathrm{com},m}(t) = \sum_{p,q \in p_{k,j,n}(t)} \frac{p_p o_{k,j,n}^m(t) x_{k,j,n}(t) M_{p,q}(t)}{R_{p,q}(t)},$$

and

$$E_{k,j,n}^{\mathrm{ccom}}(t) = \sum_{p,q \in p_{k,j,n}(t)} \frac{p_p \left(1 - \sum_{m \in \mathcal{N}} o_{k,j,n}^m(t)\right) x_{k,j,n}(t) M_{p,q}(t)}{R_{p,q}(t)}. \tag{16}$$

The energy consumption of LEO satellite m for executing task $k_{j,n}(t)$ can be given by

$$E_{k,j,n}^{\mathrm{comp},m}(t) = \varepsilon (f_{m,j})^3 D_{k,j,n}^{\mathrm{comp},m}(t), \tag{17}$$

where $\varepsilon (f_{m,j})^3$ is a function of power and computing capability [34].

Then, the energy consumption is computed as:

$$E(t) = \sum_{n \in \mathcal{N}} \sum_{j \in \mathcal{J}} \sum_{k \in \Gamma_{j,n}(t)} \sum_{m \in \mathcal{N}} \left(E_{k,j,n}^{\mathrm{com},m}(t) + E_{k,j,n}^{\mathrm{comp},m}(t) \right) + E_{k,j,n}^{\mathrm{ccom}}(t). \tag{18}$$

F. Task Arrival Prediction and Load Balancing Model

Repetitive scheduling of tasks poses challenges in satellite edge computing networks due to their highly dynamic nature and the uneven distribution of tasks. Therefore, an intelligent prediction scheme is implemented to forecast the future generation of tasks on LEO satellites and a computing load balancing model is built to effectively deal with this challenge.

In our work, an ARIMA model [35] is employed to predict future task generation trends for LEO satellites due to changes over time and space. Let W represent the prediction scale. Then, the predicted task generation state of LEO satellites in time slot t can be denoted by $\Upsilon(t) = \{\Gamma(t+1), \ldots, \Gamma(t+w), \ldots, \Gamma(t+W)\}$. We consider the remaining processing delay of tasks in the LEO satellites' queue to measure the computing load of the network. Therefore, the load balancing factor of the network can be defined as:

$$B(t) = \frac{\sum_{n \in \mathcal{N}} \left(b_n(t) - \bar{b}(t)\right)}{N}. \tag{19}$$

Here, $b_n(t)$ is the average remaining processing delay of data in the service queue on LEO satellite n, and can be given by:

$$b_n(t) = \sum_{j \in \mathcal{J}_n} \frac{c_j \left(Q_{n,j}(t) + \Lambda_{n,j}(t) + \sum_{w=1}^{W} \rho^w \Gamma_{j,n}(t+w)\right)}{|J_n| f_{n,j} v_{j,n}}, \tag{20}$$

where $\rho^w \in (0,1)$ is the weight factor. $\bar{b}(t)$ is the average computing load of the network in time slot t and can be obtained by:

$$\bar{b}(t) = \frac{\sum_{n \in \mathcal{N}} b_n(t)}{N}. \tag{21}$$

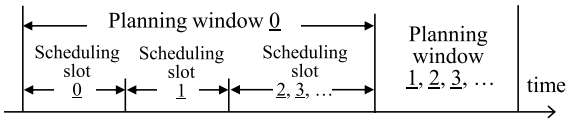


Fig. 2. Illustration of the two-timescale decision.

IV. Joint Service Deployment and Task Scheduling Framework

This section commences with a presentation of the problem statement, followed by a description of a hierarchical framework for the two-timescale joint optimization problem.

A. Problem Statement

The focus of our study is to resolve the joint service deployment and computing task scheduling problem occurring across two distinct timescales. For the small-timescale task scheduling, our aim is to make scheduling decisions for satellite tasks within scheduling time slots, while ensuring processing delay constraints, reducing the energy consumption, balancing the network load, and alleviating the packet loss rate. Adjustments to task scheduling are expected to be minimal, possibly on the order of seconds. On the other hand, for the large-timescale service deployment, our aim is to optimize the deployment decisions of services in each planning window. The focus here is to maximize the computing service capabilities for tasks and minimize the costs associated with service deployment. Alterations in service deployment within a containerized operating environment cannot be too minor, typically occurring on the order of tens of minutes. Fig. 2 illustrates the relationship between decisions at two timescales. Given the interdependence and correlation between the two problems occurring at different timescales, we establish a hierarchical optimization framework. This framework enables us to combine small-timescale task scheduling and large-timescale service deployment to jointly optimize the computing service capabilities of satellite edge computing networks.

B. Small-Timescale Computing Task Scheduling

1) CMDP Formulation: Task scheduling in satellite edge computing networks requires a balance of minimizing energy consumption, maintaining load balance, and reducing packet loss rate while simultaneously meeting long-term delay requirements. This can be modeled using the concept of CMDP, where actions, states, and costs are defined as follows.

a) State: Denote by $\mathcal{S}$ the set of possible states. The composite state $\boldsymbol{s}_t \in \mathcal{S}$ for the small-timescale computing task scheduling in time slot t can be described as:

$$\boldsymbol{s}_t = \Big\{ \{\Gamma_n(t)\}_{n \in \mathcal{N}}, \{\mathcal{Q}_{n,j}(t)\}_{n \in \mathcal{N}, j \in \mathcal{J}_n}, \{R_{n,0}(t)\}_{n \in \mathcal{N}}, \{R_{n,m}(t)\}_{n,m \in \mathcal{N}}, \Upsilon(t) \Big\}, \tag{22}$$

where $\{\Gamma_n(t)\}_{n \in \mathcal{N}}$ is the task generation state, $\{\mathcal{Q}_{n,j}(t)\}_{n \in \mathcal{N}, j \in \mathcal{J}_n}$ is the data queue state, $\{R_{n,0}(t)\}_{n \in \mathcal{N}}$ and $\{R_{n,m}(t)\}_{n,m \in \mathcal{N}}$ is the

communication state, and $\Upsilon(t)$ is the predicted task generation state.

b) Actions: In response to the current state $\boldsymbol{s}_t \in \mathcal{S}$, LEO satellites will execute task scheduling actions. The system action $\boldsymbol{a}_t \in \mathcal{A}$ during time slot t can be depicted as:

$$\boldsymbol{a}_t = \Big\{ \left\{ o^m_{k,j,n}(t) \right\}_{n,m \in \mathcal{N}, j \in \mathcal{J}, k \in \Gamma_{j,n}(t)}, \left\{ p_{k,j,n}(t) \right\}_{n \in \mathcal{N}, j \in \mathcal{J}, k \in \Gamma_{j,n}(t)} \Big\}. \tag{23}$$

We must underline that the set $\mathcal{A}$ comprises actions $\boldsymbol{a}_t \in \mathcal{A}$ that meet the following criteria. The first condition dictates that the amount of tasks scheduled cannot surpass the amount of tasks generated, that is, $\sum_{m \in \mathcal{N}} \sum_{k \in \Gamma_{j,n}(t)} o^m_{k,j,n}(t) \leq \Gamma_{n,j}(t)$. The second condition ensures that LEO satellites can only accept tasks that correspond to services deployed locally, meaning $o^m_{k,j,n}(t) > 0$ if $j \in \mathcal{J}_m$ and $v_{m,j} > 0$.

c) Cost: The objective of the small-timescale task scheduling optimization problem is to make task scheduling decisions considering the task generation state, communication state, data queue state, and predicted future task generation state. Our aim is to reduce the energy consumption of task processing, achieve load balancing, and alleviate packet loss rate in satellite edge computing networks, all while ensuring that task processing delay requirements are met. Accordingly, the immediate cost in time slot t is expressed as

$$c_t = \kappa^E E(t) + \kappa^B B(t) + \kappa^\Psi \Psi(t), \tag{24}$$

where κ^E, κ^B and κ^Ψ are performance gain coefficients. The delay constraint of service j in time slot t is computed as: $d_{j,t} = D_j(t)$

Our objective is to discover a stationary policy π, which will dynamically configure task scheduling decisions to minimize energy consumption, load imbalance, and queue overflow while ensuring delay constraints. The optimization problem can be formulated as:

$$\begin{aligned} (\mathrm{P1}): \min_{\pi} \lim_{T \to \infty} \frac{1}{T} \sum_{t=1}^{T} \mathbb{E}\left[c_t \mid \pi \right] \\ \text{s.t.} \lim_{T \to \infty} \frac{1}{T} \sum_{t=1}^{T} d_{j,t} \leq D_j^{th} \quad \forall j \in \mathcal{J}. \end{aligned} \tag{25}$$

2) Lyapunov-Based Problem Transformation: The main hurdle in solving problem (P1) involves managing long-term constraints. Hence, we leverage Lyapunov technology [36], [37] which centralizes the idea of using a delay deficit queue to represent the level of fulfillment of long-term delay constraints, thereby guiding the agent to meet these constraints. The process of problem transformation is as follows.

We initially establish delay deficit queues $\{Y_{j,t}\}_{j \in \mathcal{J}}$ for all services, whose dynamics updates by

$$Y_{j,t+1} = \left[d_{j,t} - D_j^{th} + Y_{j,t} \right]^+, j \in \mathcal{J} \tag{26}$$

where $Y_{j,t}$ represents the deviation of the true delay and the maximum tolerance delay, and we set its initial state as $Y_{i,0} = 0$. To represent the level of satisfaction with delay requirements, a Lyapunov function is introduced, represented as $L\left(Y_{j,t}\right) = \left(Y_{j,t}\right)/2$.

Then, to ensure adherence to the long-term delay constraint, the Lyapunov function should be as small as possible. Thus, we introduce a one-shot Lyapunov drift to track the change in the Lyapunov function between two successive time slots, which is represented as $\Delta(Y_{j,t}) = L(Y_{j,t+1}) - L(Y_{j,t})$. The upper limit of this drift is given as:

$$\begin{aligned} \Delta(Y_{j,t}) &= \frac{1}{2}\left(\left(Y_{j,t+1}\right)^2 - \left(Y_{j,t}\right)^2 \right) \\ &\leq \frac{1}{2}\left(\left(Y_{j,t} + d_{j,t} - D_j^{th}\right)^2 - \left(Y_{j,t}\right)^2 \right) \\ &= \frac{1}{2}\left(d_{j,t} - D_j^{th}\right)^2 + Y_{j,t}\left(d_{j,t} - D_j^{th}\right) \\ &\leq C_j + Y_{j,t}\left(d_{j,t} - D_j^{th}\right), \end{aligned} \tag{27}$$

where $C_j = D_j^{\max} - D_j^{th}$ is a constant, with $D_j^{\max}$ representing the maximum achievable delay for service j. The first inequality is a result of substituting (26), while the second inequality holds due to the constraint $d_{j,t} \leq D_j^{\max}$.

Following Lyapunov optimization theory, the above optimization objective can be simplified to minimize the drift-plus-penalty term, i.e.

$$\begin{aligned} &\sum_{j \in \mathcal{J}} \Delta\left(Y_{j,t}\right) + \varsigma \cdot c_t \\ &\quad \leq \sum_{j \in \mathcal{J}} C_j + \sum_{j \in \mathcal{J}} Y_{j,t}\left(d_{j,t} - D_j^{th}\right) + \varsigma \cdot c_t, \end{aligned} \tag{28}$$

where the inequality is a result of the upper boundary defined in (27). ς represents the weighting constant that adjusts each optimization indicator to satisfy the long-term energy consumption, load balance and queue overflow constraints.

3) Equivalent MDP: The introduction of delay deficit queues leads to the following state, action, and cost function in the equivalent MDP.

a) State: In contrast to the CMDP state, the backlog of the delay deficit queue for services ought to be included, defined as follows:

$$\hat{\boldsymbol{s}}_t = \left\{ \boldsymbol{s}_t, \{Y_{j,t}\}_{j \in \mathcal{J}} \right\}. \tag{29}$$

b) Actions: The action remains identical to that in the CMDP, denoted by $\hat{\boldsymbol{a}}_t = \boldsymbol{a}_t$.

c) Cost: The cost is adjusted to minimize the drift plus-cost, as indicated in (28), for time slot t. This is represented as follows:

$$\hat{c}_t = \varsigma \cdot c_t + \sum_{j \in \mathcal{J}} Y_{j,t}\left(d_{j,t} - D_j^{th}\right). \tag{30}$$

For brevity, the constant term $\sum_{j \in \mathcal{J}} C_j$ in (28) is omitted in the cost.

Then, problem (P1) is converted into the following MDP problem:

$$\hat{\mathbf{P}}1: \min_{\pi} \lim_{T \to \infty} \frac{1}{T} \sum_{t=1}^{T} \mathbb{E}\left[\sum_{j \in \mathcal{J}} Y_{j,t}\left(d_{j,t} - D_j^{th}\right) + \varsigma \cdot c_t \mid \pi \right]. \tag{31}$$

C. Large-Timescale Service Deployment

The scheduling performance of tasks is closely related to service deployment in the network. In satellite edge computing networks, where LEO satellites move at high speeds and computing tasks vary rapidly, it is essential to keep small task scheduling changes. Adjusting the service deployment policy during task scheduling can generate significant deployment overhead. Therefore, in the case of a fixed task scheduling policy, we further optimize the service deployment policy in the satellite edge computing network to maximize the overall network service capability within a larger planning window, while reducing the deployment overhead of services.

For each type of service, the corresponding service replica runs on LEO satellites by instantiating the container. Container instantiation can be performed in two ways: cold start and warm start [20], [38], [39]. The warm start involves reusing an existing service instance, while a cold start initiates a new service instance. The cold start process may include starting new containers, configuring the runtime environment, and deploying services, resulting in higher time and energy consumption compared to a warm start. Due to the limited resources in satellite edge computing networks, frequent service deployment is not feasible. We posit that the cold start delay induced by deploying a new replica of service $j \in \mathcal{J}$ on LEO satellite n is represented as $\omega_{n,j}$. Additionally, we acknowledge that the warm start time is insignificantly smaller compared to the cold start and task processing delays, and hence, we omit its consideration in this paper.

The queuing delay in each service queue is directly influenced by the number of service replicas deployed on LEO satellites. In response to incoming task requests, the LEO satellites dynamically adjust the deployment of their services. During each planning window, when there is an increase in requests for a particular service, the number of deployed replicas for that service is increased. Conversely, when requests for a certain service decrease, the number of deployed replicas is reduced accordingly. To determine the number of service replicas of type j that need to be cold started on LEO satellite n for the current planning window $\delta_{j,n}$, we can calculate it based on the service deployment policy for the previous planning window, denoted by $v'_{j,n}$, and the current service deployment policy for the current planning window, denoted by $v_{j,n}$. Then, $\delta_{j,n}$ can be given by

$$\delta_{j,n} = \begin{cases} v_{j,n} - v'_{j,n}, & v_{j,n} \geq v'_{j,n} \\ 0, & \text{otherwise.} \end{cases} \tag{32}$$

Hence, for each type of service j on LEO satellite n, the delay and energy consumption caused by the cold start of containers in the current planning window can be determined as follows:

$$D_n^{\text{large}} = \sum_{j \in \mathcal{J}} \omega_{n,j} \delta_{j,n} \text{ and } E_n^{\text{large}} = \sum_{j \in \mathcal{J}} \varepsilon (f_{n,j})^3 \omega_{n,j} \delta_{j,n}. \tag{33}$$

Then, we can calculate the cost of service deployment as follows:

$$\text{Cost}^{\text{large}} = \sum_{n \in \mathcal{N}} \left(D_n^{\text{large}} + E_n^{\text{large}} \right). \tag{34}$$

In satellite edge computing networks, service deployment aims to efficiently utilize limited storage resources by deploying services on-demand to meet various computing requests. Consequently, the benefits of service deployment can be defined as:

$$\text{Ben}^{\text{large}} = \sum_{n \in \mathcal{N}} \sum_{j \in \mathcal{J}} \log \Bigg(1+ \zeta_j \sum_{t \in \mathcal{T}} \Bigg(\sum_{m \in \mathcal{N}} \sum_{k \in \Gamma_{j,n}(t)} o^n_{k,j,m}(t) x_{k,j,m}(t) - \Psi_{n,j}(t) \Bigg) v_{j,n} \Bigg), \tag{35}$$

where ζ_j ($j \in \mathcal{J}$) is the weight factor.

The formulation of the large-timescale service deployment problem is as follows:

$$(\text{P2}): \max_{v_{j,n}} \text{Ben}^{\text{large}} - \iota \text{Cost}^{\text{large}} \tag{36}$$

$$\text{s.t.} \begin{cases} \sum_{j \in \mathcal{J}} v_{j,n} z_j \leq Z_n, n \in \mathcal{N} \\ v_{j,n} \in \mathbb{Z}^+, n \in \mathcal{N}, j \in \mathcal{J}. \end{cases} \tag{37}$$

where ι is the performance gain coefficient, and Z_n is the storage capability of LEO satellite n.

D. Joint Service Deployment and Task Scheduling Framework

The superior task scheduling policy from problem (P1) relies on the service deployment in satellite edge computing networks. If services are not appropriately deployed, the network may fail to achieve superior energy consumption, load balancing, and packet loss rate, potentially violating the task processing delay constraint. To achieve excellent network service performance, service deployment needs to be optimized, as stated in (P2). Hence, it is imperative to collaboratively address the two-timescale problems (P1) and (P2) for achieving the joint optimization of network performance and service performance. The aim of the joint service deployment and computing task scheduling framework is to identify superior task scheduling policies $\hat{\boldsymbol{a}}^* = \{o^{m,*}_{k,j,n}, p^*_{k,j,n} \mid n, m \in \mathcal{N}, j \in \mathcal{J}_n, k \in \Gamma_{j,n}(t)$ and superior service deployment policies $\{v^*_{j,n} \mid n \in \mathcal{N}, j \in \mathcal{J}_n\}$. In the satellite edge computing network, the ground data center will periodically collect various types of information, including downlink/inter-satellite communication conditions, computing resource and storage resource usage of LEO satellites, and the generation of computing tasks. Leveraging this collected information, the ground data center makes task scheduling decisions and service deployment decisions, subsequently transmitting these decisions to LEO satellites for execution.

V. Hierarchical Approach for Joint Optimization Problem

This section introduces a hierarchical approach to address the combined problem of service deployment and task scheduling at two timescales.

A. SAC-Based Small-Timescale Task Scheduling

To address the small-timescale task scheduling problem ($\hat{\mathbf{P}}1$), we devise a DRL algorithm grounded on SAC.

1) SAC Framework: SAC is to identify an optimal task scheduling policy $\pi(\hat{\boldsymbol{a}}|\hat{\boldsymbol{s}})$ that minimizes the expected cost. The learning agent operates at the ground and consists of two distinct components: the actor (policy) and the critic (value function). To encourage continuous exploration, the cost function incorporates an entropy term $\mathcal{H}(\pi(\hat{\boldsymbol{a}}|\hat{\boldsymbol{s}}))$ [40], [41]. Let $\mathcal{H}\left(\pi\left(\cdot|\hat{\boldsymbol{s}}\right)\right) = \mathbb{E}_{\hat{\boldsymbol{a}}}\left[-\log\pi\left(\hat{\boldsymbol{a}}|\hat{\boldsymbol{s}}\right)\right]$, for policy π, the soft Q-value function is estimated as follows:

$$Q^{\pi}(\hat{\boldsymbol{s}},\hat{\boldsymbol{a}}) = \mathbb{E}\Bigg[\sum_{t=0}^{\infty}\gamma^{t}c_{t} + \alpha\sum_{t=1}^{\infty}\gamma^{t}H\left(\pi\left(\cdot\mid\hat{\boldsymbol{s}}_{t}\right)\right)\mid\hat{\boldsymbol{s}}_{0}=\hat{\boldsymbol{s}},\hat{\boldsymbol{a}}_{0}=\hat{\boldsymbol{a}}\Bigg], \tag{38}$$

where γ is the discount factor and α is the temperature parameter. Employing fully connected DNNs, the soft Q-value function is characterized as $Q_{\boldsymbol{\theta}}(\hat{\boldsymbol{s}},\hat{\boldsymbol{a}})$ with parameter $\boldsymbol{\theta}$, and the policy is characterized as $\pi_{\boldsymbol{\psi}}(\hat{\boldsymbol{a}}|\hat{\boldsymbol{s}})$ with parameter $\boldsymbol{\psi}$.

2) Algorithm Design: In line with the aforementioned framework, we develop a SAC-based small-timescale task scheduling algorithm which comprises numerous training episodes, where every episode consists of several steps. In our work, a single episode corresponds to one planning window and a single step aligns with the scheduling time slot.

- Initialization: The parameters of soft Q-value functions $Q_{\boldsymbol{\theta}_l}\left(\hat{\boldsymbol{s}}_t,\hat{\boldsymbol{a}}_t\right)$, $\forall l \in \{1,2\}$, policies $\pi_{\boldsymbol{\psi}}\left(\hat{\boldsymbol{a}}|\hat{\boldsymbol{s}}\right)$, and the replay memory $\mathcal{M}_r$ are initially set.

- Experience sampling: During each time slot, the learning agent observes the network state $\hat{\boldsymbol{s}}_t$ and combines this state with the policy $\pi_{\boldsymbol{\psi}}(\hat{\boldsymbol{a}}_t|\hat{\boldsymbol{s}}_t)$ to generate an action $\hat{\boldsymbol{a}}_t$. With the selected action, the agent receives a feedback cost $\hat{c}_t$ and transitions to the next state $\hat{\boldsymbol{s}}_{t+1}$. This experience is represented as a tuple $\{\hat{\boldsymbol{s}}_t,\hat{\boldsymbol{a}}_t,\hat{c}_t,\hat{\boldsymbol{s}}_{t+1}\}$, capturing the performed action, state transition, and feedback cost. The experience is subsequently put into in the experience replay memory $\mathcal{M}_r$.

- Parameter updating: The actor and critic networks are refreshed by randomly selecting a mini-batch $\mathcal{M}_b$ of tuples $\left\{\hat{\boldsymbol{s}}_t^b,\hat{\boldsymbol{a}}_t^b,\hat{c}_t^b,\hat{\boldsymbol{s}}_{t+1}^b\right\}$ from $\mathcal{M}_r$.

a. Critic updating: To circumvent the overestimation of state values, we design two distinct critic networks with parameters $\boldsymbol{\theta}_1$ and $\boldsymbol{\theta}_2$. We regard the smaller Q-value between $Q_{\boldsymbol{\theta}_1}$ and $Q_{\boldsymbol{\theta}_2}$, as the authentic Q-value. We separately update $\boldsymbol{\theta}_l$, $\forall l \in \{1,2\}$ for each evaluation critic network to minimize the loss function $L(\boldsymbol{\theta}_l)$, which can be expressed as follows:

$$\mathcal{L}\left(\boldsymbol{\theta}_l\right) = \mathbb{E}_{\mathcal{M}_b}\left[\frac{1}{2}\left(Q_{\boldsymbol{\theta}_l}\left(\hat{\boldsymbol{s}}_t,\hat{\boldsymbol{a}}_t\right) - y_t\right)^2\right]. \tag{39}$$

Here, y_t represents the target soft value that employs the target parameter $\bar{\boldsymbol{\theta}}_l$, $\forall l \in \{1,2\}$, which is computed as:

$$y_t = c_t + \gamma\Big(\min_{l=1,2} Q_{\bar{\boldsymbol{\theta}}_l}\left(\hat{\boldsymbol{s}}_{t+1},\hat{\boldsymbol{a}}_{t+1}\right) - \alpha\log\left(\pi_{\boldsymbol{\psi}}\left(\hat{\boldsymbol{a}}_{t+1}|\hat{\boldsymbol{s}}_{t+1}\right)\right)\Big). \tag{40}$$

Then, we update the parameters $\boldsymbol{\theta}_l$, $\forall l \in \{1,2\}$ by computing the gradients of $\mathcal{L}\left(\boldsymbol{\theta}_l\right)$. To maintain the stability of learning, we use a soft-updating method to update the target critic network parameters $\bar{\boldsymbol{\theta}}_l$ from $\boldsymbol{\theta}_l$, which is expressed as:

$$\bar{\boldsymbol{\theta}}_l = \epsilon\boldsymbol{\theta}_l + (1-\epsilon)\bar{\boldsymbol{\theta}}_l, \forall l = 1,2 \tag{41}$$

where $\epsilon \in (0,1)$ is the update factor.

b. Actor updating: The parameters of the actor is updated by minimizing the following equation:

$$J(\boldsymbol{\psi}) = \mathbb{E}_{\mathcal{M}_b}\left[\mathrm{D}_{\mathrm{KL}}\left(\pi_{\boldsymbol{\psi}}\left(\cdot|\hat{\boldsymbol{s}}_t\right)\middle\|\frac{\exp\left(\min_{l=1,2} Q_{\boldsymbol{\theta}_l}\left(\hat{\boldsymbol{s}}_t,\cdot\right)\right)}{Z_{\boldsymbol{\theta}_l}\left(\hat{\boldsymbol{s}}_t\right)}\right)\right] \tag{42}$$

where the Kullback-Leibler (KL) divergence $\mathrm{D}_{\mathrm{KL}}(p||q)$ quantifies dissimilarity between the distributions p and q. The partition function $Z_{\boldsymbol{\theta}_l}\left(\hat{\boldsymbol{s}}_t\right)$ is intractable and does not influence the gradient of the new policy. As such, the above KL-divergence can be transformed as:

$$J(\boldsymbol{\psi}) = -\mathbb{E}_{\mathcal{M}_b}\Bigg[\mathop{\mathbb{E}}_{\hat{\boldsymbol{a}}_{t+1}\sim\pi_{\boldsymbol{\psi}}}\Big[\min_{l=1,2} Q_{\bar{\boldsymbol{\theta}}_l}\left(\hat{\boldsymbol{s}}_{t+1}^m,\hat{\boldsymbol{a}}_{t+1}^m\right) - \alpha\log\left(\pi_{\boldsymbol{\psi}}\left(\hat{\boldsymbol{a}}_{t+1}^m|\hat{\boldsymbol{s}}_{t+1}\right)\right)\Big]\Bigg]. \tag{43}$$

Subsequently, the policy parameters $\boldsymbol{\psi}$ are updated by determining the gradient of $J(\boldsymbol{\psi})$.

c. Temperature updating: The temperature parameter can be self-adjusted by calculating the gradient in relation to the following objective function:

$$J\left(\alpha\right) = \mathbb{E}_{\pi_{\boldsymbol{\theta}}}\left[-\alpha\log\left(\pi_{\boldsymbol{\theta}}\left(\hat{\boldsymbol{a}}_t|\hat{\boldsymbol{s}}_t\right)\right) - \alpha\bar{H}\right], \tag{44}$$

where $\bar{H}$ is the value of target entropy.

Our proposed SAC-based DRL algorithm is a constrained learning approach due to the limited number of LEO satellites and state spaces within the satellite edge computing network. Additionally, our algorithm employs a replay memory strategy, allowing it to store essential historical state information. At each step, the satellite edge computing network gathers environmental states information through observation, taking corresponding actions in the pursuit of minimizing the immediate drift plus-cost. As a result, our proposed optimization algorithm ensures convergence. The learning process of the proposed SAC-based task scheduling algorithm alternates between soft critic updating and soft actor updating until it converges to a policy with maximum entropy. The steps for this algorithm are outlined in Algorithm 1, where the computational efficiency primarily hinges on the calculation of parameters for convolutional neural networks (CNNs) [42]. A convolution layer's computational complexity is denoted by $W_c = \mathcal{O}\left(\sum_{n_c=1}^{N_c}(s_e)^2(s_k)^2)\lambda_{n_c}\lambda_{n_c-1}\right)$, with N_c representing the number of convolutional layers, s_e as the feature map size, s_k as the kernel size, and λ as the number of filters. A fully-connected layer's computational complexity is given as $W_f = \mathcal{O}\left(\sum_{n_c=1}^{N_c}\mu_{n_c}\mu_{n_c-1}\right)$, with μ_{n_c} indicating the number of neural units in fully-connected

Algorithm 1 SAC-Based Small-Timescale Task Scheduling Algorithm

Input: T, γ, $|\mathcal{M}_r|$, $|\mathcal{M}_b|$.
Initialize $\boldsymbol{\theta}_l (l = 1, 2)$, $\overline{\boldsymbol{\theta}}_l = \boldsymbol{\theta}_l$, $\boldsymbol{\psi}$, α, $\mathcal{M}_r$;
for *each episode* $ep = 1, 2, \ldots, Ep$ **do**
  Set the initial network state to $\mathbf{s}_0$;
  for *each time slot* $t = 1, 2, \ldots, T$ **do**
    ********Experience sampling********;
    Acquire the system state $\hat{\boldsymbol{s}}_t = \{\boldsymbol{s}_t, Y_{j,t} \mid j \in \mathcal{J}\}$;
    Obtain task scheduling decision $\hat{\boldsymbol{a}}_t = \{o^m_{k,j,n}(t), p_{k,j,n}(t) \mid n, m \in \mathcal{N}, j \in \mathcal{J}_n, k \in \Gamma_{j,n}(t)\}$ with policy $\pi_{\boldsymbol{\psi}}(\hat{\boldsymbol{a}}|\hat{\boldsymbol{s}})$;
    Schedule tasks to other LEO satellites according to $\hat{\boldsymbol{a}}_t$;
    Observe immediate cost $\hat{c}_t$ and obtain the next state $\hat{\boldsymbol{s}}_{t+1}$ through message passing;
    Record the four tuples $\{\hat{\boldsymbol{s}}_t, \hat{\boldsymbol{a}}_t, \hat{c}_t, \hat{\boldsymbol{s}}_{t+1}\}$ and update the oldest experiences in the replay memory $\mathcal{M}_r$;
    ********Parameter updating********;
    Draw a minibatch of $\mathcal{M}_b$ experiences randomly from $\mathcal{M}_r$;
    Compute the target Q-value y_t using (40);
    Update the parameters $\boldsymbol{\theta}_l$ of soft Q-value function by minimizing the loss function defined in (39);
    Update the parameters $\boldsymbol{\psi}$ of task scheduling policy using the gradient defined in (43);
    Adjust temperature parameter α by calculating the gradient as detailed in (44);
    Update target networks using (41);
  end
end
Output: Task scheduling policy $\hat{\boldsymbol{a}}^*$.

layer μ. Consequently, Algorithm 1's total computational complexity is $\mathcal{O}(EpT(W_c + W_f))$. The proposed SAC-based algorithm necessitates the training of numerous neural networks, potentially consuming resources and time. In practice, since the SAC-based algorithm is an offline algorithm [43], [44], it can be pre-trained offline at the ground data center. Then, the trained algorithm can then be used directly to quickly make scheduling decisions in an online manner.

B. AOS-Based Large-Timescale Service Deployment

This section aims to address the large-timescale service deployment problem. A service deployment algorithm based on AOS is proposed [45]. This algorithm is largely inspired by certain principles of quantum mechanics and the conduct of quantum atomic models, taking into account the characteristics of electrons around the nucleus, and has the advantages of strong optimization ability and fast convergence speed. The steps of the proposed algorithm are as follows.

Algorithm 2 AOS-Based Large-Timescale Service Deployment Algorithm

Input: maximum number of iterations, weight factors.
Initialize candidate solutions;
Repair the initial candidate solutions;
Evaluate the energy state for initial candidate solution;
Determine the candidate with the lowest energy level L_E;
while *iteration* $< X_A$ **do**
  Create the number of imaginary layers, denoted by U;
  Arrange the candidate solutions in descending order;
  Allocate candidate solutions across imaginary layers according to the PDF;
  for $u = 1, \ldots, U$ **do**
    Establish binding state B^u_S and binding energy B^u_E of the uth layer;
    Identify the candidate in the uth layer with the lowest energy level, denoted by L^u_E; **for** $i = 1, \ldots, i_u$ **do**
      Determine the photon rate P_R;
      if $\varphi \geq P_R$ **then**
        if $E^{i_u,u}_A \geq B^u_E$ **then**
          Update candidate solutions by (48);
        end
        else
          Update candidate solutions by (49);
        end
      end
      else
        Update candidate solutions by (50);
      end
    end
  end
  Repair the candidate solutions;
  Update candidate with the lowest energy level L_E;
end
Output: Service deployment policy $\boldsymbol{V}^* = \arg\min_i E^i_A$.

- Initialize candidate solutions: The candidate solutions for service deployment is defined as follows:

$$\boldsymbol{V} = \left[\boldsymbol{V}^1, \ldots, \boldsymbol{V}^i, \ldots, \boldsymbol{V}^I\right]^\top, \tag{45}$$

where I is the number of candidate solutions (electrons) inside the search space and $\boldsymbol{V}^i = [v^i_{1,1}, \ldots, v^i_{1,N}, \ldots, v^i_{J,1}, \ldots, v^i_{J,N}]$. Then, the set of candidate solutions can be initialized in a random manner.

- Repair candidate solutions: The candidate solutions are constrained by (37). Therefore, to make the generated candidate solutions valid, the candidate solutions need to be repaired. In this work, $v^i_{j,n}$ with a value greater than 0 is randomly selected, and the -1 operation is performed on it until the candidate solution satisfies the constraint (37).

- Construct AOS energy functions: Based on a quantum model of the atom, electrons have corresponding energy states $\boldsymbol{E}_A = \left[E_A^1, \ldots, E_A^i, \ldots, E_A^I\right]^\top$ that correspond to the objective function values for candidate solutions to the service deployment problem. Candidate solutions with superior objective function values are analogous to electrons at lower energy levels, while candidate solutions with poorer objective function values are akin to electrons at higher energy levels.

- Divide imaginary layers: The placement of electrons around the nucleus is established based on the electron probability density map. Therefore, the proposed algorithm considers creating imaginary layers around the kernel and utilizes the probability density function (PDF) to position candidate solutions within these layers. Candidate solutions are sorted in an ordinal manner, with those exhibiting superior objective function values assigned higher ranks. Assuming that there are U imaginary layers, the candidate solutions and energy of the u-th ($u = 1 \ldots, U$) imaginary layer can be denoted by $\boldsymbol{V} = \left[\boldsymbol{V}^{1,u}, \ldots, \boldsymbol{V}^{i,u}, \ldots, \boldsymbol{V}^{I,u}\right]^\top$ and $\boldsymbol{E}_A = [E_A^{1,u}, \ldots, E_A^{i,u}, \ldots, E_A^{I,u}]^\top$.

- Calculate the binding state, binding energy, and the lowest energy level: The optimal candidate solution within each imaginary layer is considered as the electron having the lowest energy level L_E^u in that specific layer. Furthermore, out of all the candidate solutions, the one with the superior objective function value is perceived as the electron at the atom's lowest energy level, L_E. The energy required to detach an electron from its shell is referred to as binding energy. The binding state and binding energy of the u-th imaginary layer can be characterized as follows:

$$B_S^u = \frac{\sum_{i_u=1}^{I_u} \boldsymbol{V}^{i_u,u}}{I_u} \text{ and } B_E^u = \frac{\sum_{i_u=1}^{I_u} E_A^{i_u,u}}{I_u}, \tag{46}$$

where I_u represents the total count of candidate solutions present in the uth imaginary layer. Then, we can define the binding state and binding energy of all these candidate solutions as follows:

$$B_S = \frac{\sum_i^I \boldsymbol{V}^{i,u}}{I} \text{ and } B_E = \frac{\sum_{i=1}^I E_A^{i,u}}{I}. \tag{47}$$

- Update candidate solutions: The rate of photon, represented as P_R, signifies the likelihood of a photon's influence on an electron being considered. Each electron generates a randomly distributed number, φ, that falls within the range of $(0, 1)$. If $\varphi > P_R$, then the movement of electrons across different layers surrounding the nucleus takes into account photons, based on their emission and absorption properties. If the energy of a candidate solution surpasses the binding energy of a particular imaginary layer, a photon is emitted. Let $\tilde{\boldsymbol{V}}^{i_u,u}$ represent the updated electron, it can be updated by:

$$\tilde{\boldsymbol{V}}^{i_u,u} = \boldsymbol{V}^{i_u,u} + \frac{\chi_{i_u}^{\text{emit}} \times \left(\varpi_{i_u}^{\text{emit}} \times L_E - \eta_{i_u}^{\text{emit}} \times B_S\right)}{u}. \tag{48}$$

If the energy of the candidate solution is lower than the binding energy of a particular imaginary layer, a photon is absorbed. Then, $\tilde{\boldsymbol{V}}^{i_u,u}$ can be updated as follows:

$$\tilde{\boldsymbol{V}}^{i_u,u} = \boldsymbol{V}^{i_u,u} + \chi_{i_u}^{\text{absorb}} \times \left(\varpi_{i_u}^{\text{absorb}} \times L_E^u - \eta_{i_u}^{\text{absorb}} \times B_S^u\right). \tag{49}$$

If $\varphi < P_R$, electrons absorb or emit energy through particle/magnetic field interactions. Then, $\tilde{\boldsymbol{V}}^{i_u,u}$ can be updated as follows:

$$\tilde{\boldsymbol{V}}^{i_u,u} = \boldsymbol{V}^{i_u,u} + \chi_{i_u}^{\text{particle-magnetic}}, \tag{50}$$

where χ_{i_u}, ϖ_{i_u} and η_{i_u} are evenly distributed on $(0, 1)$, which are weight factors. Given the constrained number of LEO satellites and the limited search space within the satellite edge computing network, our designed AOS-based heuristic algorithm converges to an excellent solution through iterations. Problem (P2) is an integer programming problem with exponential computational complexity. Let X_A represent the maximum number of iterations of Algorithm 2. Then, the total computational complexity of Algorithm 2 can be denoted by $\mathcal{O}(X_A I)$, reflecting a significant reduction in complexity.

Algorithm 3 Hierarchical Algorithm for Joint Optimization

Input: Initialized the satellite edge computing environment, various parameters of Algorithm 1 and Algorithm 2.

repeat
- $g = g + 1$;
- Obtain the service deplyment in satellite edge computing networks accrording to $\{v_{j,n}^{(g-1)} \mid n \in \mathcal{N}, j \in \mathcal{J}_n\}$;
- With$\{v_{j,n}^{(g-1)} \mid n \in \mathcal{N}, j \in \mathcal{J}_n\}$, determine the stationary task scheduling policy $\hat{\boldsymbol{a}}^{(g)}$ by Algorithm 1;
- Given $\hat{\boldsymbol{a}}^{(g)}$, determine the service deployment policy $\{v_{j,n}^{(g)} \mid n \in \mathcal{N}, j \in \mathcal{J}_n\}$ in iteration g by Algorithm 2;

until $\hat{\boldsymbol{a}}^{(g)} \approx \hat{\boldsymbol{a}}^{(g-1)}$;

Output: A set of optimal solutions for task scheduling $\hat{\boldsymbol{a}}^* = \{o_{k,j,n}^{m,*}, p_{k,j,n}^* \mid n, m \in \mathcal{N}, j \in \mathcal{J}_n, k \in \Gamma_{j,n}(t)$ and service deployment $\{v_{j,n}^* \mid n \in \mathcal{N}, j \in \mathcal{J}_n\}$.

C. Hierarchical Solution for Joint Optimization

Drawing on the individual convergence of problems (P1) and (P2), we introduce a two-timescale hierarchical optimization framework that provides iterative solutions to problems (P1) and (P2) for achieving joint task scheduling and service deployment decisions, as shown in Fig. 3. Algorithm 3 presents a detailed description of our hierarchical approach. Specifically, after the first iteration of solving (P1), the task scheduling policy $\hat{\boldsymbol{a}}^{(1)}$ is obtained and fed back into the service deployment problem (P2) to update the service deployment policy $\{v_{j,n}^{(1)} \mid n \in \mathcal{N}, j \in \mathcal{J}_n\}$ for maximizing the overall network service capabilities and reducing service deployment

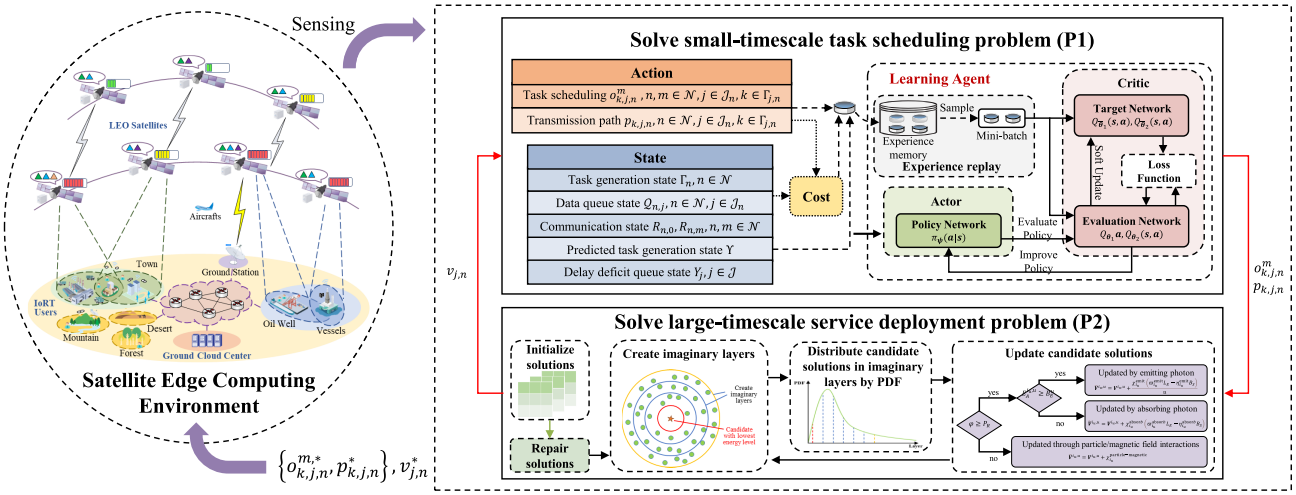


Fig. 3. Illustration of the two-timescale hierarchical optimization framework.

overhead under the current scheduling policy. Then, the second iteration commences by sequentially solving (P1) and (P2) once again to determine $\hat{\boldsymbol{a}}^{(2)}$ and $\{v_{j,n}^{(2)} \mid n \in \mathcal{N}, j \in \mathcal{J}_n\}$. This process is repeated until the task scheduling policy $\hat{\boldsymbol{a}}^{(g)}$ obtained after multiple iterations is close to $\hat{\boldsymbol{a}}^{(g-1)}$. The output from (P1) and (P2) is a set of superior solutions for task scheduling $\hat{\boldsymbol{a}}^* = \{o_{k,j,n}^{m,*}, p_{k,j,n}^* \mid n, m \in \mathcal{N}, j \in \mathcal{J}_n, k \in \Gamma_{j,n}(t)$ and service deployment $\{v_{j,n}^* \mid n \in \mathcal{N}, j \in \mathcal{J}_n\}$ that jointly optimize the satellite edge computing network and the computing service capabilities. After achieving convergence, which is typically reached after X_H iterations, the total computational complexity of Algorithm 3 can be calculated as $\mathcal{O}\left(X_H\left(EpT(W_c + W_f) + X_A I\right)\right)$.

VI. Simulation Results and Discussions

This section details the simulation results that demonstrate the performance of our proposed framework designed for joint service deployment and task scheduling in satellite edge computing.

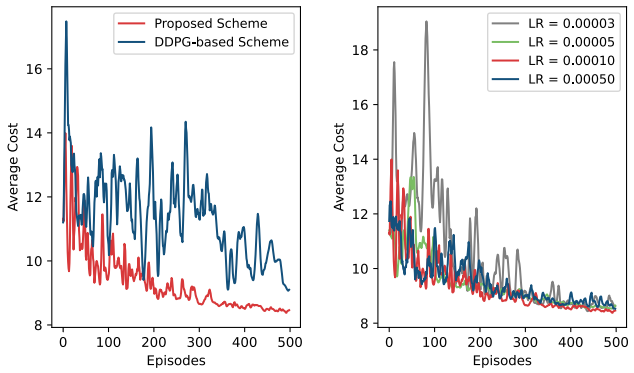
A. Simulation Setup

The experimental parameters have been established in accordance with the guidelines of previous research [20], [33], [35], [46]. The environment is constructed using the Pytorch framework. Our scenario considers a satellite edge computing network composed of 20 LEO satellites, with the longitude and latitude data corresponding to the Walker-Delta constellation. The assumed transmission power for the LEO satellites stands at 5 W, and the total noise temperature for the system is set to 25 dB K. The transmitting and receiving antenna gains for LEO satellites in the inter-satellite link are 20 dB. Additionally, the transmission gain for LEO satellites in the downlink is 37 dB. The ratio of the required received energy per bit to the noise density is 9.6 dB, and the link margin is 2500 km. Each scheduling slot and planning window is timed at 1 sec and 10 min, respectively. The number of service types is 3. Each LEO satellite has a total storage capacity of 32 GB, with the required storage capacity for each service deployed on LEO satellites uniformly distributed within 100 MB to 500 MB. For this study, we chose image processing tasks where each image varies in size from 10 KB to 30 MB. The amount of CPU cycles required to process each bit of a computing task is uniformly distributed within 200 to 600 cycles. The cold start delay, which is the time taken to launch services on LEO satellites, varies between 0.15 sec and 0.85 sec. The processing capability of services deployed on LEO satellites ranges from 0.1 to 0.5 Gcycles/sec. The performance gain coefficients κ^E, κ^B and κ^Ψ of small-timscale task scheduling are set to 0.5, 0.1, and 1, respectively. The performance gain coefficient ι of large-timscale service deployment is set to 0.38.

For the proposed SAC-based small-timescale task scheduling algorithm, numerical results are compiled after 600 learning episodes. The learning rate (LR) and the discount rate are set to 0.0001 and 0.85, respectively. To compute the loss function, a minibatch consisting of 256 tuples is randomly selected from a replay memory, which holds the last $10,000$ experiences. For the proposed AOS-based large-timescale service deployment algorithm, the size of candidate solutions is set to 200, and the photon rate is set to 0.1.

We conducted a series of experiments to obtain accurate and reliable results. To assess the effectiveness of our proposed scheme, we compared it against the following benchmark schemes:

- *DDPG-based scheme*: at every small-timescale decision period, the DDPG method [47] is used to select task scheduling policies.
- *All-cloud scheme*: at every small-timescale decision period, all tasks are scheduled to the ground cloud center for processing.
- *Local scheme*: at every small-timescale decision period, all tasks are processed locally on LEO satellites.
- *PSO-based scheme*: at every large-timescale decision period, the particle swarm optimization (PSO) method [48] is adopted to select service deployment policies.
- *Greedy scheme*: selection of service deployment and task scheduling decisions based on greedy ideas.



(a) Different DRL frameworks (b) Different learning rates

Fig. 4. Convergence performance of the proposed scheme under different DRL frameworks.

- *Stationary deployment scheme*: service deployment policies remain constant across planning windows and is not subject to dynamic adjustments.

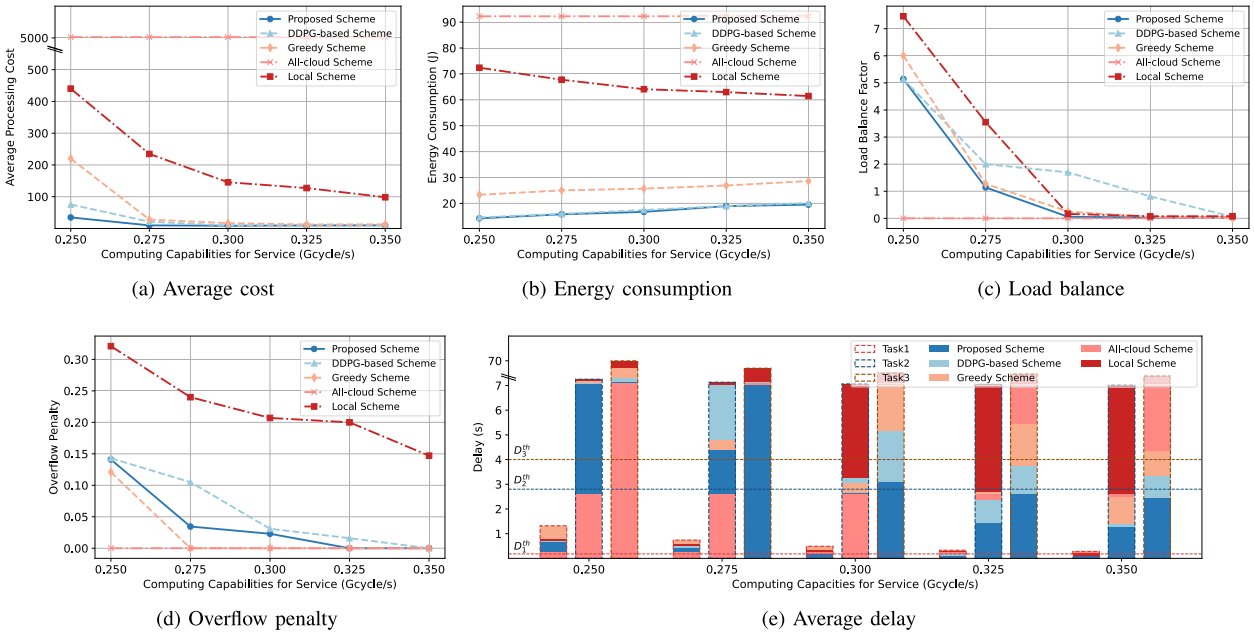
B. Simulation Results

Based on the defined parameters, simulations are performed to evaluate the efficiency of our proposed scheme. In Fig. 4, we first analyze the convergence performance of the proposed scheme on small timescales. The DDPG-based task scheduling scheme is adopted here as the benchmark scheme for comparison. As demonstrated in Fig. 4(a), there is a gradual decline in the average cost (per time slot) of small-timescale task scheduling under the two schemes, which eventually converges to an estimated solution. Our proposed scheme's remarkable superiority over DDPG-based schemes is due to the adoption of a maximum entropy regularized stochastic strategy rather than a deterministic one, which prevents the overfitting of the value function and consequent fragility. Additionally, the influence of LR on the scheme's effectiveness is explored in Fig. 4(b). It is observed that an escalated LR propels a swifter convergence rate for the proposed scheme. It is crucial to acknowledge that an excessively fast convergence might lead to the solution getting stuck in a local optimum instead of reaching a global one, emphasizing the need for a carefully balanced LR. Consequently, after rigorous experiments, we decided to fix LR at 0.0001 for the following simulations due to its balanced convergence speed and remarkable learning stability.

Subsequently, we gauge the performance of our proposed scheme by juxtaposing it with several task scheduling schemes on small timescales. In Fig. 5, we scrutinize the effectiveness of our proposed scheme in comparison to the DDPG-based scheme, greedy scheme, all-cloud scheme, and local scheme, given the average computing capabilities of service replicas on LEO satellites. Evaluation metrics utilized include average cost, energy consumption, load balancing, overflow penalty, and delay associated with small-timescale task scheduling. As demonstrated in the figure, the all-cloud scheme, which schedules all computing tasks to the ground cloud center, remains constant across all metrics, regardless of the computing capabilities on LEO satellites. The energy consumption of the remaining four schemes consistently increases with the enhanced computing capabilities on LEO satellites, while the load balancing, overflow penalty, and delay metrics decrease with the escalation of computing capabilities on LEO satellites. The reasoning behind these results is as follows. The enhancement of computing capabilities impacts the energy consumed in task processing due to the corresponding increase in LEO satellites' computing power. Moreover, as the computing capabilities on LEO satellites amplifies, more tasks can be processed within a single scheduling slot, leading to a reduction in load and packet loss and a decrease in task waiting and processing delay. Among the five schemes, the optimal performance consistently resides with the proposed scheme, trailed by the DDPG-based scheme and the greedy scheme. In contrast, the local scheme and the all-cloud scheme exhibit comparatively subpar performance. When the computing capabilities on LEO satellites are at 0.3 Gcycle/s, the average cost of the proposed scheme is approximately 93.1%, 51.0%, 0.16%, and 5.86% of the DDPG-based scheme, greedy scheme, all-cloud scheme, and local scheme, respectively. Concurrently, the average delay of the proposed scheme is about 68.5%, 56.2%, 59.9%, and 39.8% of the DDPG-based scheme, greedy scheme, all-cloud scheme, and local scheme, respectively.

In Fig. 6, we delve into an analysis of the convergence performance of our proposed scheme over large timescales. As a benchmark for comparison, the PSO-based service deployment scheme is employed. This figure depicts the overall utility of large-timescale service deployment under two scenarios. This utility gradually increases with the number of iterations, ultimately converging towards the estimated solution. Notably, our proposed scheme exhibits a faster convergence time compared to the PSO-based service deployment scheme, accompanied by significantly superior utility attainment. The accelerated convergence and enhanced utility underscore the effectiveness of our proposed scheme in optimizing large-timescale service deployment.

In Fig. 7, we gauge the performance of our proposed method by juxtaposing it with a variety of large timescale service deployment schemes. We assess the effectiveness of our proposed method in comparison with PSO-based scheme and greedy scheme, considering various average storage capacities of LEO satellites. The evaluation metrics encompass average utility, benefits, and costs of service deployment. As depicted in the figure, the utility, benefit, and cost of the proposed scheme and the PSO-based scheme rise incrementally as the storage capacity of LEO satellites increases. This can be attributed to the fact that an enhanced LEO satellite storage capacity allows for more extensive service deployment within the network, subsequently leading to a rise in deployment costs. Simultaneously, the upsurge in the parallel rate of computing tasks escalates the number of tasks that can be processed and scheduled within a single planning window, thereby augmenting the benefits of service deployment within the satellite edge computing network. The average utility of the greedy scheme exhibits a pattern of first rising and then falling. This is because the scheme allocates storage resources and deploys corresponding services in a greedy manner based



(a) Average cost (b) Energy consumption (c) Load balance

(d) Overflow penalty (e) Average delay

Fig. 5. Performance of the proposed scheme and the benchmark task scheduling scheme on small timescales for the given average computing capabilities of service replicas on LEO satellites.

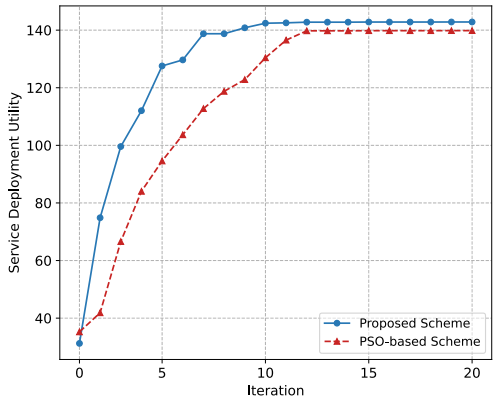
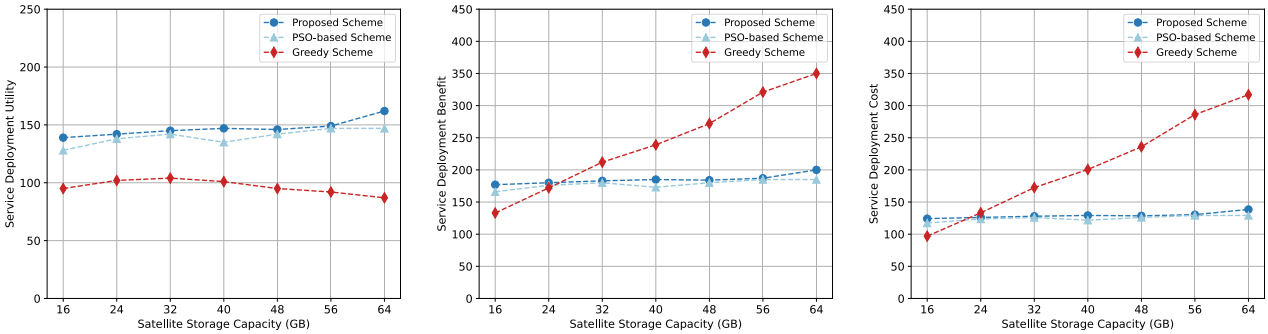


Fig. 6. Convergence performance of the proposed large-timescale service deployment scheme and the benchmark PSO-based service deployment scheme.

on service requests. As the storage capacity of LEO satellites increases, deployment costs rise at a faster rate than the benefits, leading to this trend. Among the three schemes, our proposed scheme consistently outperforms the rest, followed closely by the PSO-based scheme, while the greedy scheme yields the least favorable results. When the storage capacity of LEO satellites is at 32 GB, the average utility of our proposed scheme is boosted by nearly 2.1% and 39.4% in comparison to the PSO-based scheme and greedy scheme, respectively.

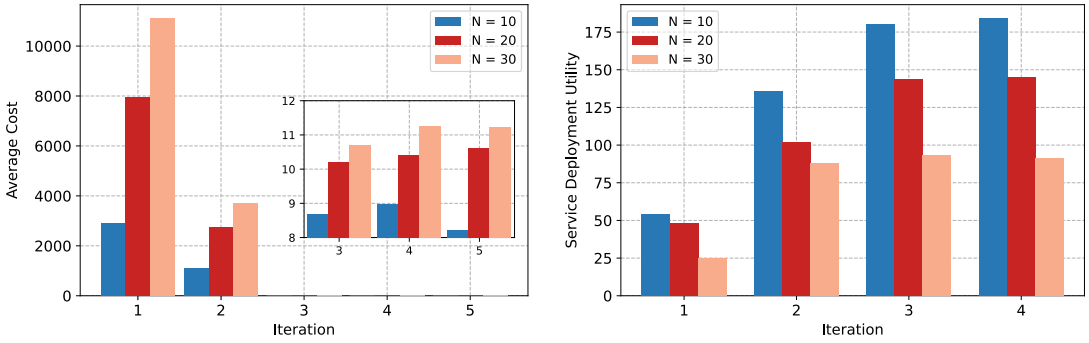
We conducted simulations to evaluate the performance of our proposed hierarchical joint optimization scheme. In Fig. 8, we examine the convergence performance of the hierarchical joint optimization scheme. The initial service deployment policy is randomly generated, and in each subsequent iteration, the task scheduling policy is determined based on the service deployment policy obtained in the previous iteration. Fig. 8(a) illustrates that as the number of iterations increases, the average cost of small-timescale task scheduling gradually decreases and converges rapidly. Fig. 8(b) demonstrates that with an increasing number of iterations, the effectiveness of large-timescale service deployment gradually improves and converges quickly. Furthermore, Fig. 8 provides insights into the performance of the proposed hierarchical joint optimization scheme under different numbers of participating LEO satellites. As the network scale expands, the average cost of task scheduling gradually increase due to the growing number of tasks that need processing. As an illustration, upon the convergence of 30 LEO satellites, the total scheduling cost increases by 19.6% and 13.5%, respectively, in comparison to scenarios involving 10 LEO satellites and 20 LEO satellites. Conversely, as the number of nodes increases, the utility of service deployment exhibits a declining trend. This is attributed to the network scale expansion causing a broader and more uneven distribution of tasks, resulting in increased deployment costs. Upon achieving convergence in the scenario with 30 LEO satellites, the reduction is 48.3% and 33.5%, respectively, compared to scenarios involving 10 and 20 LEO satellites.

The average task scheduling cost of the proposed hierarchical joint optimization scheme over 6 consecutive planning windows is depicted in Fig. 9. It's noteworthy that the arrival of tasks varies in each planning window, with the total data amount of tasks in each planning window being $[54.6, 55.8, 57.6, 58.2, 67.8, 69]$ GB, respectively. In this analysis, we utilize a stationary deployment scheme as a benchmark for comparison. The figure illustrates how the average task scheduling cost of the two scenarios varies across different planning windows, each with different task arrivals. The task scheduling cost is affected by the quantity of tasks arriving within the planning window. Increased task arrivals lead to higher scheduling costs, as the average cost of task scheduling gradually escalates with the growing number of tasks that need processing. Moreover, the proposed hierarchical joint optimization scheme exhibits



(a) Overall utility (b) Deployment benefit (c) Deployment cost

Fig. 7. Performance of the proposed scheme and the benchmark service deployment scheme on large timescales for the given average storage capabilities of LEO satellites.



(a) Average Task Scheduling Cost (b) Service Deployment Utility

Fig. 8. Convergence performance of the proposed hierarchical joint optimization scheme.

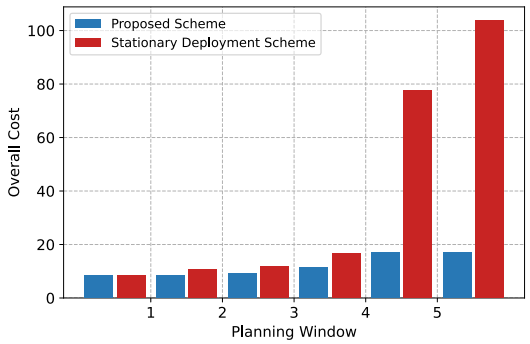


Fig. 9. Performance of the average task scheduling cost of the proposed scheme and the stationary deployment scheme over continuous planning windows.

the capability to dynamically adjust the service deployment policy between planning windows, showcasing superior performance compared to the stationary deployment scheme. In the fourth planning window, the proposed hierarchical joint optimization scheme achieves a 39.4% reduction in the average task scheduling cost compared to the stationary deployment scheme.

In conclusion, the numerical results highlight the effectiveness of the proposed scheme in achieving joint service deployment and task scheduling through iterative processes. Notably, on smaller timescales, the proposed scheme exhibits faster convergence and superior practicality compared to the DDPG-based task scheduling scheme. It outperforms benchmark task scheduling schemes in various aspects, including average cost, energy consumption, load balancing, overflow penalty, and delay, across different parameter settings. Furthermore, on larger timescales, the proposed scheme demonstrates superior performance in terms of overall utility, benefit, and deployment cost when compared to PSO-based, and greedy-based benchmark service deployment schemes. Such improvements signify the ability of the proposed scheme to effectively guarantee the QoS of computing tasks and improve the overall service performance of the satellite edge computing network through optimized service deployment and task scheduling.

VII. Conclusion

This paper established a two-timescale framework to achieve asynchronous optimization of service deployment and task scheduling for satellite edge computing networks. The small-timescale task scheduling was formulated as a CMDP to optimize the energy consumption, load balancing and packet loss of networks while ensuring the long-term delay. The Lyapunov-based transformation technique was employed to deal with the delay constraints. The large-timescale service deployment was modeled as an integer programming problem to optimize the computing service capacity of networks and reduce the cost of service deployment. Due to the correlation between the problems of two timescales, a hierarchical solution based on the SAC framework and the AOS approach was

constructed to iteratively find the superior solution. Finally, we conducted extensive experiments to validate the effectiveness and superiority of our proposed scheme. We anticipate that computing tasks will become more intricate with the advent of new application types. Large computing tasks will be decomposed, and sub-tasks will exhibit tight coupling. The scheduling of computing tasks necessitates a more thorough examination of sub-task dependencies and correlations, aspects that will be addressed in future work.

REFERENCES

[1] X. Lin, S. Cioni, G. Charbit, N. Chuberre, S. Hellsten, and J. Boutillon, "On the path to 6G: Embracing the next wave of low earth orbit satellite access," *IEEE Commun. Mag.*, vol. 59, no. 12, pp. 36–42, Dec. 2021.

[2] Z. Xiao et al., "LEO satellite access network (LEO-SAN) towards 6G: Challenges and approaches," *IEEE Wireless Commun.*, early access, Dec. 5, 2022, doi: 10.1109/MWC.011.2200310.

[3] T. Chen et al., "Efficient uplink transmission in ultra-dense LEO satellite networks with multiband antennas," *IEEE Commun. Lett.*, vol. 26, no. 6, pp. 1373–1377, Jun. 2022.

[4] D. Zhou, M. Sheng, Y. Wang, J. Li, and Z. Han, "Machine learning-based resource allocation in satellite networks supporting Internet of Remote Things," *IEEE Trans. Wireless Commun.*, vol. 20, no. 10, pp. 6606–6621, Oct. 2021.

[5] T. Chen et al., "Learning-based computation offloading for IoRT through Ka/Q-band satellite-terrestrial integrated networks," *IEEE Internet Things J.*, vol. 9, no. 14, pp. 12056–12070, Jul. 2022.

[6] W. C. Ng et al., "Resource optimization for UAV-assisted wireless power charging enabled hybrid coded edge computing network," *IEEE Trans. Mobile Comput.*, vol. 23, no. 3, pp. 2022–2038, Mar. 2024.

[7] Q. Tang et al., "Collective deep reinforcement learning for intelligence sharing in the Internet of Intelligence-Empowered edge computing," *IEEE Trans. Mobile Comput.*, vol. 22, no. 11, pp. 6327–6342, Nov. 2023.

[8] Q. Luo, S. Hu, C. Li, G. Li, and W. Shi, "Resource scheduling in edge computing: A survey," *IEEE Commun. Surveys Tuts.*, vol. 23, no. 4, pp. 2131–2165, 4th Quart., 2021.

[9] R. Xie, Q. Tang, Q. Wang, X. Liu, F. R. Yu, and T. Huang, "Satellite-terrestrial integrated edge computing networks: Architecture, challenges, and open issues," *IEEE Netw.*, vol. 34, no. 3, pp. 224–231, May/Jun. 2020.

[10] X. Gao, R. Liu, and A. Kaushik, "Virtual network function placement in satellite edge computing with a potential game approach," *IEEE Trans. Netw. Service Manag.*, vol. 19, no. 2, pp. 1243–1259, Jun. 2022.

[11] B. Zhu, S. Lin, Y. Zhu, and X. Wang, "Collaborative hyperspectral image processing using satellite edge computing," *IEEE Trans. Mobile Comput.*, vol. 23, no. 3, pp. 2241–2253, Mar. 2024.

[12] Y. Zhang, C. Chen, L. Liu, D. Lan, H. Jiang, and S. Wan, "Aerial edge computing on orbit: A task offloading and allocation scheme," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 1, pp. 275–285, Jan./Feb. 2023.

[13] S. Wan, J. Hu, C. Chen, A. Jolfaei, S. Mumtaz, and Q. Pei, "Fair-hierarchical scheduling for diversified services in space, air and ground for 6G-dense Internet of Things," *IEEE Trans. Netw. Sci. Eng.*, vol. 8, no. 4, pp. 2837–2848, Oct. 2021.

[14] P. Cassará, A. Gotta, M. Marchese, and F. Patrone, "Orbital edge offloading on mega-LEO satellite constellations for equal access to computing," *IEEE Commun. Mag.*, vol. 60, no. 4, pp. 32–36, Apr. 2022.

[15] X. Zhang et al., "Energy-efficient computation peer offloading in satellite edge computing networks," *IEEE Trans. Mobile Comput.*, early access, Apr. 25, 2023, doi: 10.1109/TMC.2023.3269801.

[16] X. Cao et al., "Edge-assisted multi-layer offloading optimization of LEO satellite-terrestrial integrated networks," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 2, pp. 381–398, Feb. 2023.

[17] J. C. McDowell, "The low Earth orbit satellite population and impacts of the SpaceX starlink constellation," *Astrophysical J. Lett.*, vol. 892, no. 2, p. 36, Apr. 2020.

[18] J. Radtke, C. Kebschull, and E. Stoll, "Interactions of the space debris environment with mega constellations—Using the example of the OneWeb constellation," *Acta Astronautica*, vol. 131, pp. 55–68, Feb. 2017.

[19] J. Liu, G. Li, Z. H. Zhu, M. Liu, and X. Zhan, "Automatic orbital maneuver for mega-constellations maintenance with electrodynamic tethers," *Aerosp. Sci. Technol.*, vol. 105, Oct. 2020, Art. no. 105910.

[20] Q. Tang et al., "Distributed task scheduling in serverless edge computing networks for the Internet of Things: A learning approach," *IEEE Internet Things J.*, vol. 9, no. 20, pp. 19634–19648, Oct. 2022.

[21] Z. Zhang, W. Zhang, and F.-H. Tseng, "Satellite mobile edge computing: Improving QoS of high-speed satellite-terrestrial networks using edge computing techniques," *IEEE Netw.*, vol. 33, no. 1, pp. 70–76, Jan. 2019.

[22] C. Ding, J.-B. Wang, M. Cheng, M. Lin, and J. Cheng, "Dynamic transmission and computation resource optimization for dense LEO satellite assisted mobile-edge computing," *IEEE Trans. Commun.*, vol. 71, no. 5, pp. 3087–3102, May 2023.

[23] N. Cheng et al., "Space/aerial-assisted computing offloading for IoT applications: A learning-based approach," *IEEE J. Sel. Areas Commun.*, vol. 37, no. 5, pp. 1117–1129, May 2019.

[24] B. Yang, X. Cao, J. Bassey, X. Li, and L. Qian, "Computation offloading in multi-access edge computing: A multi-task learning approach," *IEEE Trans. Mobile Comput.*, vol. 20, no. 9, pp. 2745–2762, Sep. 2021.

[25] H. Zhang, R. Liu, A. Kaushik, and X. Gao, "Satellite edge computing with collaborative computation offloading: An intelligent deep deterministic policy gradient approach," *IEEE Internet Things J.*, vol. 10, no. 10, pp. 9092–9107, May 2023.

[26] Z. Ji, S. Wu, and C. Jiang, "Cooperative multi-agent deep reinforcement learning for computation offloading in digital twin satellite edge networks," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 11, pp. 3414–3429, Nov. 2023.

[27] T.-D. Nguyen, E.-N. Huh, and M. Jo, "Decentralized and revised content-centric networking-based service deployment and discovery platform in mobile edge computing for IoT devices," *IEEE Internet Things J.*, vol. 6, no. 3, pp. 4162–4175, Jun. 2019.

[28] P. Wang, J. Xu, M. Zhou, and A. Albeshri, "Budget-constrained optimal deployment of redundant services in edge computing environment," *IEEE Internet Things J.*, vol. 10, no. 11, pp. 9453–9464, Jun. 2023.

[29] B. Li et al., "READ: Robustness-oriented edge application deployment in edge computing environment," *IEEE Trans. Services Comput.*, vol. 15, no. 3, pp. 1746–1759, May/Jun. 2022.

[30] C. Wang, F. R. Yu, C. Liang, Q. Chen, and L. Tang, "Joint computation offloading and interference management in wireless cellular networks with mobile edge computing," *IEEE Trans. Veh. Technol.*, vol. 66, no. 8, pp. 7432–7445, Aug. 2017.

[31] C. Liang, Y. He, F. R. Yu, and N. Zhao, "Enhancing QoE-aware wireless edge caching with software-defined wireless networks," *IEEE Trans. Wireless Commun.*, vol. 16, no. 10, pp. 6912–6925, Oct. 2017.

[32] W. Wu, P. Yang, W. Zhang, C. Zhou, and X. Shen, "Accuracy-guaranteed collaborative DNN inference in industrial IoT via deep reinforcement learning," *IEEE Trans. Ind. Informat.*, vol. 17, no. 7, pp. 4988–4998, Jul. 2021.

[33] D. Zhou, M. Sheng, R. Liu, Y. Wang, and J. Li, "Channel-aware mission scheduling in broadband data relay satellite networks," *IEEE J. Sel. Areas Commun.*, vol. 36, no. 5, pp. 1052–1064, May 2018.

[34] M. Merluzzi, N. D. Pietro, P. Di Lorenzo, E. C. Strinati, and S. Barbarossa, "Discontinuous computation offloading for energy-efficient mobile edge computing," *IEEE Trans. Green Commun. Netw.*, vol. 6, no. 2, pp. 1242–1257, Jun. 2022.

[35] C. B. A. Satrio, W. Darmawan, B. U. Nadia, and N. Hanafiah, "Time series analysis and forecasting of coronavirus disease in Indonesia using ARIMA model and PROPHET," *Proc. Comput. Sci.*, vol. 179, pp. 524–532, Jan. 2021.

[36] Y. Li, S. Xia, M. Zheng, B. Cao, and Q. Liu, "Lyapunov optimization-based trade-off policy for mobile cloud offloading in heterogeneous wireless networks," *IEEE Trans. Cloud Comput.*, vol. 10, no. 1, pp. 491–505, Jan. 2022.

[37] A. Asheralieva and D. Niyato, "Game theory and Lyapunov optimization for cloud-based content delivery networks with device-to-device and UAV-enabled caching," *IEEE Trans. Veh. Technol.*, vol. 68, no. 10, pp. 10094–10110, Oct. 2019.

[38] C. Cicconetti, M. Conti, A. Passarella, and D. Sabella, "Toward distributed computing environments with serverless solutions in edge systems," *IEEE Commun. Mag.*, vol. 58, no. 3, pp. 40–46, Mar. 2020.

[39] T. Rausch, A. Rashed, and S. Dustdar, "Optimized container scheduling for data-intensive serverless edge computing," *Future Gener. Comput. Syst.*, vol. 114, pp. 259–271, Jan. 2021.

[40] W. Zhang et al., "Optimizing federated learning in distributed industrial IoT: A multi-agent approach," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 12, pp. 3688–3703, Dec. 2021.
[41] Q. Tang, F. R. Yu, R. Xie, A. Boukerche, T. Huang, and Y. Liu, "Internet of Intelligence: A survey on the enabling technologies, applications, and challenges," *IEEE Commun. Surveys Tuts.*, vol. 24, no. 3, pp. 1394–1434, 3rd Quart., 1394.
[42] Y. Liu, H. Wang, M. Peng, J. Guan, and Y. Wang, "An incentive mechanism for privacy-preserving crowdsensing via deep reinforcement learning," *IEEE Internet Things J.*, vol. 8, no. 10, pp. 8616–8631, May 2021.
[43] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 1861–1870.
[44] N. C. Luong et al., "Applications of deep reinforcement learning in communications and networking: A survey," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 4, pp. 3133–3174, 4th Quart., 2019.
[45] M. Azizi, "Atomic orbital search: A novel metaheuristic algorithm," *Appl. Math. Model.*, vol. 93, pp. 657–683, May 2021.
[46] X. Li, J. Wan, H.-N. Dai, M. Imran, M. Xia, and A. Celesti, "A hybrid computing solution and resource scheduling strategy for edge computing in smart manufacturing," *IEEE Trans. Ind. Informat.*, vol. 15, no. 7, pp. 4225–4234, Jul. 2019.
[47] T. P. Lillicrap et al., "Continuous control with deep reinforcement learning," 2015, *arXiv:1509.02971*.
[48] F. Marini and B. Walczak, "Particle swarm optimization (PSO). A tutorial," *Chemometric Intell. Lab. Syst.*, vol. 149, pp. 153–165, Dec. 2015.

Qinqin Tang received the Ph.D. degree from the Beijing University of Posts and Telecommunications (BUPT), Beijing, China, in 2022. She is currently a Post-Doctoral Research Fellow with BUPT. Her current research interests include satellite networks, mobile edge networks, traffic offloading, and resource management and allocation.

Renchao Xie (Senior Member, IEEE) received the Ph.D. degree from the Beijing University of Posts and Telecommunications, China, in 2012. Currently, he is a Professor with the School of Information and Communication Engineering, Beijing University of Posts and Telecommunications. His research interests include edge intelligence, computing power networks, integrated networking and computing, and the Industrial Internet of Things. He has served as the technical program committee (TPC) co-chair for numerous conferences. He has also served for several journals as an associate editor or a reviewer.

Zeru Fang is currently pursuing the master's degree with the State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications, China. His current research interests include satellite-terrestrial integrated networks, edge/cloud computing, and software defined networking (SDN).

Tao Huang (Senior Member, IEEE) received the B.S. degree in communication engineering from Nankai University, Tianjin, China, in 2002, and the M.S. and Ph.D. degrees in communication and information system from the Beijing University of Posts and Telecommunications, Beijing, China, in 2004 and 2007, respectively. He is currently a Professor with the Beijing University of Posts and Telecommunications. His current research interests include network architecture, network artificial intelligence, routing and forwarding, and network virtualization.

Tianjiao Chen received the Ph.D. degree from the Beijing University of Posts and Telecommunications, Beijing, China, in 2022. He is currently a Research Fellow with the China Mobile Research Institute. His research interests include machine learning, network virtualization, and software defined networking (SDN).

Ran Zhang received the Ph.D. degree from the Beijing University of Posts and Telecommunications (BUPT), Beijing, China, in 2021. He is currently an Associate Researcher with the School of Information and Communication Engineering, BUPT. His research interests include satellite communications, named data networking, and artificial intelligence.

F. Richard Yu (Fellow, IEEE) is currently a Professor with Carleton University, Canada. His research interests include connected autonomous vehicles, artificial intelligence, and security. He serves on the editorial boards for several journals, including a Lead Series Editor for IEEE TRANSACTIONS ON VEHICULAR TECHNOLOGY, IEEE TRANSACTIONS ON GREEN COMMUNICATIONS AND NETWORKING, and IEEE COMMUNICATIONS SURVEYS & TUTORIALS. He is a fellow of IET, a Distinguished Lecturer of IEEE Vehicular Technology (VT) Society and Communications Society, and the Vice President (Membership) of the IEEE VT Society.